

Politechnika Poznańska
Wydział Informatyki
Instytut Informatyki

Praca dyplomowa magisterska

**ANALIZA I OPTYMALIZACJA MECHANIZMU KOMUNIKACJI
MIĘDZY APLIKACJAMI W SYSTEMACH ROZPROSZONYCH
PRZY UŻYCIU MODELU RPC**

Marcin Piniarski, 122498

Promotor
dr hab. inż. Anna Kobusińska

Poznań, 2019 r.

Tutaj przychodzi karta pracy dyplomowej;
oryginał wstawiamy do wersji dla archiwum PP, w pozostałych kopiach wstawiamy ksero.

Spis treści

1	Wstęp	1
1.1	Motywacja	1
1.2	Temat	1
1.3	Cel i zakres pracy	2
2	Model systemu	3
3	Problemy komunikacji między aplikacjami w modelu RPC	5
3.1	Przesyłanie niepotrzebnych pól zasobu	5
3.2	Żądanie wielu zasobów	6
3.3	Problem N+1 zapytań	6
4	Istniejące rozwiązania	7
5	Proponowane mechanizmy rozwiązujące problemy komunikacji między aplikacjami	9
5.1	Żądanie dostosowane do konkretnego przypadku użycia.	9
5.2	Projekcja zasobu przy pomocy parametru „fields”.	10
5.3	Połączenie zasobu powiązanego przy pomocy parametru „include”.	10
5.4	Agregacja zapytań	10
5.5	Jednoczesne zastosowanie ww. mechanizmów	10
5.6	Optymalizacja na poziomie logiki biznesowej i warstwy dostępu do danych	11
6	Wybrane przypadki użycia i scenariusze testowe	12
6.1	Przypadek testowy 1: Przesyłanie niepotrzebnych pól zasobu	12
6.2	Przypadek testowy 2: Problem N+1 zapytań	12
6.3	Przypadek testowy 3: Żądanie wielu zasobów	13
7	Wybrane technologie	14
8	Implementacja	15
8.1	Biblioteka kucik-api	15
8.2	Implementacja mechanizmu projekcji zasobu	17
8.3	Implementacja mechanizmu połączenia zasobu powiązanego	18
8.4	Implementacja mechanizmu agregacji zapytań	19
8.5	Implementacja aplikacji testowej	20
9	Testy	22
10	Wyniki	24
10.1	Analiza testów wydajnościowych	24
10.1.1	Przesyłanie niepotrzebnych pól zasobu	24
10.1.2	Żądanie wielu zasobów	26
10.1.3	Problem N+1 zapytań	27
10.2	Analiza kosztów tworzenia i utrzymania systemu	30
10.2.1	Żądanie dostosowane do konkretnego przypadku użycia.	30
10.2.2	Projekcja zasobu przy pomocy parametru „fields”.	31
10.2.3	Połączenie zasobu powiązanego przy pomocy parametru „include”.	31
10.2.4	Agregacja zapytań	31

10.3 Wnioski	31
11 Podsumowanie i kierunek dalszych prac	32
Literatura	33

Rozdział 1

Wstęp

1.1 Motywacja

Wybór odpowiedniego modelu komunikacji między aplikacjami jest kluczowy przy tworzeniu systemów i rzutuje na szybkość ich powstawania, wydajność oraz łatwość utrzymania. Jest to ważna decyzja projektowa, której podjęcie wymaga wnikliwej analizy projektowanego systemu, dostępnych technologii oraz kompetencji programistów biorących udział w jego tworzeniu. Wybór ten wpływa bezpośrednio na jakość i złożoność tworzonych aplikacji, przede wszystkim może powodować potrzebę pisania większej ilości kodu w celu obsłużenia danego modelu oraz wykonywania większej liczby modyfikacji w miarę rozwoju systemu.

W zależności od modelu może wystąpić znaczne powiązanie aplikacji ze sobą, co powoduje częstą potrzebę zmiany kodu w jednej z nich wraz ze zmianami w drugiej. Zdarza się, że rozwój aplikacji klienta wymaga ciągłych zmian w funkcjonalności serwera, co utrudnia tworzenie i utrzymanie systemu. Silne powiązanie aplikacji rzutuje również na skalowalność systemu.

Model komunikacji może także być powodem problemów wydajnościowych, spowalniać działanie systemu oraz zwiększać zapotrzebowanie na zasoby takie jak moc obliczeniowa, pamięć RAM czy transfer danych przez internet.

Ważne jest, by projektując system wybrać taki model, który zminimalizuje koszty jego tworzenia i utrzymania, ograniczy zależność komponentów oraz pozwoli na rozwiązywanie problemów wydajnościowych.

1.2 Temat

Istnieje wiele modeli komunikacji, jakie można zastosować w systemach rozproszonych. Dzielią się one na modele oparte na komendach (remote invocation) oraz zdarzeniach (indirect communication) [MB11]. W modelach opartych na komendach klient wysyła żądania do serwera w celu uzyskania danych potrzebnych do dalszego przetwarzania lub w celu zlecenia wykonania pewnych akcji przez serwer. Powoduje to względnie silne powiązanie aplikacji ze sobą. Z kolei w modelach opartych na zdarzeniach klient zgłasza do serwera wystąpienie pewnej sytuacji, na którą serwer może w odpowiedni sposób zareagować. Dzięki temu aplikacje nie są sprzężone, ponieważ klient nie oczekuje na odpowiedź ze strony serwera ani nie spodziewa się konkretnej obsługi danego zdarzenia. Z drugiej strony, w takim modelu utrudnione jest pobieranie danych z serwera, obsługa błędów oraz zachowanie spójności. Model oparty na komendach jest powszechniej stosowany i bardziej intuicyjny dla programisty, ponieważ w wielu systemach konieczne jest przysyłanie dużej ilości danych o zasobach i częste wysyłanie przez klienta żądań o nowe dane.

Głównym wyzwaniem w przypadku modeli opartych na komendach jest projektowanie API (Application Programming Interface), które specyfikuje, w jaki sposób jedna aplikacja może komunikować się z inną. API określa jakie żądania są możliwe ze strony klienta, jakich parametrów może on użyć oraz jakie struktury danych są przysyłane. Na projektowanie API składają się wybranie odpowiedniego zestawu możliwych żądań, odpowiednie ich uporządkowanie, określenie ich składni i struktury. Istnieje wiele standardów projektowania API. Do najbardziej popularnych należą REST [Mas11], GraphQL [Bie18] oraz RPC [BN84]. Standardy te definiują zasady i wytyczne używane przy projektowaniu API, skupiając się na rozwiązaniu problemów oraz zapewnieniu konkretnych możliwości związanych z komunikacją między aplikacjami.

RPC (Remote Procedure Call) jest modelem, w którym struktura API odzwierciedla procedury i funkcje z kodu aplikacji serwerowej. Z punktu widzenia aplikacji klienta żądanie do serwera przypomina wywołanie lokalnej procedury lub funkcji, w rzeczywistości jednak odbywa się komunikacja między aplikacjami [BN84]. Takie podejście umożliwia proste odwzorowanie kodu aplikacji bezpośrednio na API, co ułatwia projektowanie API, przyspiesza tworzenie systemu i ogranicza ilość potrzebnego do napisania kodu. W praktyce obsługa komunikacji po stronie zarówno aplikacji klienta, jak i serwera może być wykonywana dynamicznie (przy użyciu takich mechanizmów jak refleksja [DM95] i programowanie generyczne [DRJ05]) lub przez wygenerowany automatycznie kod [BN84]. Zastosowanie RPC umożliwia też mniej kosztowną migrację z architektury monolitycznej do architektury mikroservisów, ponieważ wywołania lokalne można w łatwy sposób zastąpić przez odpowiadające im wywołania zdalne.

1.3 Cel i zakres pracy

Celem pracy jest implementacja biblioteki obsługującej komunikację przy pomocy modelu RPC, identyfikacja problemów wydajnościowych pojawiających się przy zastosowaniu takiego modelu oraz propozycja i implementacja mechanizmów mających na celu rozwiązanie tych problemów przy jednoczesnej dbałości o szybkość tworzenia i łatwość utrzymania systemu. Stworzona biblioteka umożliwia definicję API aplikacji w modelu RPC w sposób deklaratywny, dostarczając generycznych implementacji proponowanych w pracy mechanizmów optymalizujących komunikację. Użycie biblioteki pozwala na zminimalizowanie nakładu pracy potrzebnego na definicję API tworzonej aplikacji przy zachowaniu dbałości o wysoką wydajność oraz niski koszt tworzenia i utrzymania.

W ramach pracy zostały wykonane testy wydajnościowe mające na celu wykazanie efektywności proponowanych mechanizmów. Scenariusze testowe zostały dobrane w taki sposób, by odzwierciedlały realistyczne przypadki użycia, ze zwróceniem uwagi na ilość pobieranych danych oraz ich złożoność. Testy zostały przeprowadzone na danych produktu GoKoan [gok], aplikacji e-learningowej działającej na hiszpańskim rynku. Została również dokonana analiza potencjalnego wpływu zastosowanych mechanizmów na szybkość tworzenia i łatwość utrzymania systemu.

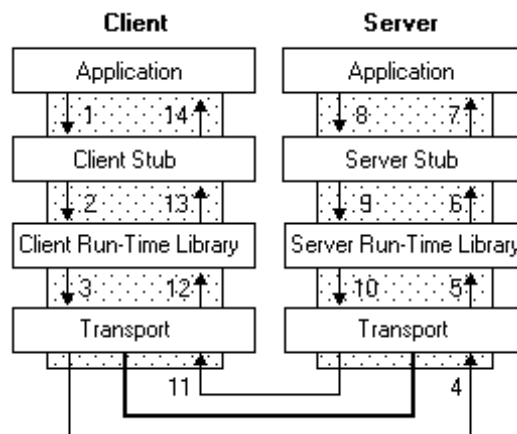
Struktura pracy jest następująca. W rozdziale 2 przedstawiono model systemu, w ramach którego dokonano analizy i implementacji opisanych w pracy. Przedstawiono przyjęte założenia i opisano model RPC. Rozdział 3 jest poświęcony problemom komunikacji występujących w systemach rozproszonych dla przyjętego modelu RPC. Rozdział 4 zawiera przegląd istniejących rozwiązań przedstawionych problemów. W rozdziale 5 zaproponowano mechanizmy mające służyć rozwiązaniu problemów komunikacji. Rozdział 6 opisuje wybrane przypadki użycia API na potrzeby przeprowadzenia analizy i testów wydajnościowych. W rozdziale 7 zawarty został opis technologii użytych przy implementacji. Rozdział 8 poświęcony jest implementacji biblioteki, zaproponowanych mechanizmów oraz aplikacji testowej. W rozdziale 9 przedstawiono metodykę wykonanych testów, opisano ich założenia oraz przebieg. Rozdział 10 zawiera opis wyników testów wydajnościowych oraz analizę mechanizmów pod kątem kosztów tworzenia i utrzymania systemu. Rozdział 11 stanowi podsumowanie pracy, zawiera wnioski oraz propozycję dalszego rozwoju omawianych zagadnień.

Rozdział 2

Model systemu

Rozpatrywane w pracy zagadnienia mają zastosowanie w kontekście systemów rozproszonych, w których między węzłami systemu dochodzi do komunikacji. Istnieje wiele mechanizmów wymiany komunikatów między węzłami w systemie rozproszonym. Niniejsza praca zakłada przyjęcie modelu RPC, mającego zastosowanie dla komunikacji między aplikacjami napisanymi w językach programowania wysokiego poziomu. Idea RPC jest oparta na obserwacji, że wywoływanie podprogramów jest szeroko stosowanym i dobrze rozumianym mechanizmem transferu danych oraz wykonywania akcji w ramach systemów niebędących częścią systemu rozproszonego. Podprogramy są podstawową jednostką komunikacji między częściami kodu w ramach algorytmu. RPC jest próbą zastosowania podobnego mechanizmu dla komunikacji między węzłami w systemach rozproszonych. Pozwala to na uzyskanie znacznego stopnia transparentności oraz intuicyjności.

Podstawowym zastosowaniem RPC jest komunikacja w architekturze klient-serwer. Gdy aplikacja klienta wywołuje procedurę zdalną, odpowiedni komunikat zawierający dane o wywołaniu, jest przesyłany do aplikacji serwerowej, po czym klient oczekuje na odpowiedź. Serwer analizuje otrzymany komunikat i wywołuje odpowiednią procedurę lokalnie, przekazując do niej określone parametry. Wynik procedury jest przesyłany w formie komunikatu do klienta, którego otrzymanie powoduje zakończenie procedury zdalnej z otrzymanym wynikiem. Obrazuje to poniższy schemat:



RYSUNEK 2.1: Schemat działania RPC [rpc]

Obsługa RPC po stronie klienta oraz serwera odbywa się dynamicznie (Client/Server Run-Time Library) oraz przy pomocy wygenerowanego kodu (Client/Server Stub). W zależności od użytej technologii oraz implementacji, elementy te mogą wykonywać różny zakres przetwarzania. W niektórych przypadkach niektóre z nich mogą więc nie występować. W czasie oczekiwania klienta na otrzymanie odpowiedzi od serwera, aplikacja klienta może zostać zawieszona lub kontynuować swoje działanie w zależności od użytych mechanizmów przetwarzania równoległego. Możliwa jest więc implementacja zarówno komunikacji blokującej, jak i nieblokującej. Użyty format komunikatów oraz mechanizm ich transportu mogą także się różnić w zależności od implementacji.

Model RPC może zostać zastosowany również w innych architekturach systemów rozproszonych i modelach komunikacji. Wówczas RPC jest użyty jako podstawa do bardziej skomplikowanych modeli. Może zostać rozwinęty do architektury peer-to-peer [Sch01], ponieważ API może zostać

zdefiniowane po każdej ze stron komunikacji. RPC może być również użyty jak podstawa do komunikacji w modelu rozgłaszania, jeśli wszystkie węzły w systemie zdefiniują w API obsługę żądania służącego do odebrania rozgłaszanej wiadomości. Podobnie możliwa jest implementacja modelu Publikowanie/Subskrybowanie [OZK07], poprzez zdefiniowanie w API aplikacji publikującej żądania pozwalającego na dokonanie subskrypcji, oraz zdefiniowanie w API aplikacji subskrybujących żądania pozwalającego na odebranie publikowanej wiadomości. Przy pomocy podstawowego modelu RPC pozwalającego na synchroniczną komunikację, możliwe jest zaimplementowanie komunikacji asynchronicznej. Wówczas serwer odpowiada natychmiastowo na zapytanie klienta, lecz po obsłudze żądania przesyła odpowiedź do aplikacji klienta, która w swoim API ma zdefiniowane żądanie obsługujące przyjęcie wyniku wysłanego wcześniej zapytania.

Rozdział 3

Problemy komunikacji między aplikacjami w modelu RPC

Wydażność komunikacji między aplikacjami zależy zarówno od zastosowanych mechanizmów przesyłu danych, ich prezentacji oraz kodowania, jak i od projektu API, możliwych żądań klienta i obsługiwanych parametrów.

Ważne jest wybranie takich mechanizmów, aby komunikacja przebiegała możliwie jak najszybciej i do wymiany koniecznych informacji przesyłana była jak najmniejsza ilość danych. W tej pracy założono zastosowanie komunikacji przy pomocy protokołu HTTP w wersji 1.1 [FGM⁺99] używając formatu wymiany danych JSON [Shi10]. Jest to popularny wybór dla tworzonych współcześnie systemów. Istnieje wiele alternatywnych technologii [NPRI09][Ste14][Fet11], lecz zagadnienie to nie jest tematem niniejszej pracy. W celu zmniejszenia rozmiaru przesyłanych danych możliwe jest użycie mechanizmów kompresji oraz buforowania dostępnych w protokole HTTP. W niniejszej pracy dokonano założenia, że powyższe mechanizmy nie są używane, gdyż utrudniłoby to analizę skuteczności proponowanych mechanizmów.

Kluczową kwestią dla komunikacji między aplikacjami jest specyfikacja możliwych żądań klienta i ich parametrów. Problemy w komunikacji między aplikacjami pojawiają się, gdy klient API nie jest w stanie precyzyjnie wyrazić przy pomocy dostępnych żądań i parametrów dokładnie tego, co potrzebuje. Prowadzi to do dwóch możliwych sytuacji: overfetching'u oraz underfetching'u.

Overfetching występuje, gdy w wyniku żądania klient otrzymuje więcej danych, niż jest mu faktycznie potrzebne. Wówczas część danych jest przesyłana nadmiarowo, co wpływa negatywnie na wydajność systemu. Zazwyczaj serwer domyślnie przesyła największy możliwy zbiór danych dostępny dla konkretnego zasobu. Klient musi pobrać cały taki zbiór, aby uzyskać dostęp do potrzebnych składowych.

Underfetching z kolei polega na otrzymaniu przez klienta niewystarczającej ilości danych w wyniku pojedynczego żądania, przez co musi on wykonywać dodatkowe zapytania w celu uzyskania wszystkich potrzebnych informacji. Utworzenie i wysłanie większej liczby zapytań wiąże się z dodatkowym kosztem. Często powoduje też skomplikowanie kodu aplikacji klienta, ponieważ wymaga obsługi każdej odpowiedzi serwera z osobna.

Poniżej znajduje się opis konkretnych problemów, na których skupiono się w tej pracy i które są przykładami problemów overfetching'u oraz underfetching'u

3.1 Przesyłanie niepotrzebnych pól zasobu

Żądania klienta często zwracają instancje zasobów, których struktura jest wyspecyfikowana w API i jest stała dla danego zasobu. Domyślnym działaniem serwera jest przesłanie całej struktury do klienta. Częstym przypadkiem jest, że klient nie potrzebuje wszystkich pól danego zasobu, ale w celu uzyskania tych mu potrzebnych, zmuszony jest pobrać cały zasób.

Szczególnie często taka sytuacja występuje, gdy z tego samego API korzysta zarówno aplikacja przeglądarkowa, jak i mobilna. Aplikacje mobilne z reguły wyświetlają mniejszą liczbę informacji w ramach widoku z uwagi na ograniczenia wielkości wyświetlacza, przez co reszta pól zasobu jest przesyłana niepotrzebnie. Dla takich aplikacji ograniczenie rozmiaru przesyłanych wiadomości do minimum ma szczególne znaczenie, ponieważ urządzenia mobilne mogą mieć większe ograniczenia co do użycia internetu, mocy obliczeniowej oraz dostępnej pamięci.

Skala problemu rośnie wraz z różnicą między ilością danych dostępnych dla danego zasobu oraz ilością faktycznie potrzebnych informacji. Zwiększa się ona również wraz z liczbą przesyłanych w jednej odpowiedzi instancji zasobu.

3.2 Żądanie wielu zasobów

W wielu przypadkach klient w celu dalszego przetwarzania potrzebuje pobrać dane kilku zasobów, a do każdego z nich w API dostęp jest możliwy przez osobne żądanie. Z tego powodu zmuszony jest wysłać kilka osobnych zapytań do serwera, zamiast uzyskać wszystkie dane w wyniku jednego zapytania.

3.3 Problem $N+1$ zapytań

Zasoby często są ze sobą powiązane, w taki sposób, że jeden zasób jest wartością pola innego zasobu. W reprezentacji przesłanej do klienta w takim polu jest zazwyczaj przesyłany identyfikator powiązanego zasobu, do którego dostęp jest możliwy przy pomocy innego żądania API. Klient często potrzebuje uzyskać dostęp zarówno do danych danego zasobu, jak i do zasobów z nim połączonych. W takim scenariuszu jest zmuszony dla każdego z powiązanych zasobów wykonać dodatkowe żądanie. Jest to szczególny przypadek problemu żądania wielu zasobów.

W sytuacji, gdy klient pobiera listę instancji danego zasobu, a dla każdej z nich potrzebuje uzyskać dostęp do zasobu powiązanego, konieczne jest w ogólności wykonanie $N+1$ zapytań, gdzie N to liczba uzyskanych instancji. Liczba koniecznych zapytań jest więc proporcjonalna do liczby danych uzyskanych z serwera. Może to prowadzić do konieczności wykonania bardzo wielu żądań przez klienta, co znacząco wpłynie na wydajność systemu. Dodatkowo problem może się szybko nasilić wraz ze wzrostem liczby powiązanych z każdą instancją zasobów potrzebnych klientowi.

Rozdział 4

Istniejące rozwiązania

Opisane problemy komunikacji między aplikacjami są często spotykane i w wielu przypadkach stanowią ważną przyczynę problemów wydajnościowych. Z tego powodu zostały podjęte próby ich rozwiązania.

Popularnym narzędziem, stworzonym z myślą o rozwiązaniu problemów `underfetching`'u i `overfetching`'u jest GraphQL, język zapytań API stworzony przez Facebook [HP17]. Głównym założeniem GraphQL jest definicja struktury zwracanych danych po stronie klienta. Poniżej znajduje się przykładowe zapytanie klienta:

```
1 {  
2   book(id: "book-1"){  
3     id  
4     name  
5     pageCount  
6     author {  
7       firstName  
8       lastName  
9     }  
10  }  
11  genres{  
12    id  
13    name  
14  }  
15 }
```

LISTING 4.1: Przykładowe zapytanie GraphQL

Zawiera ono listę żądań API, które w tym przypadku odzwierciedlają odpowiednie zasoby. Dla każdego z zasobów przekazywana jest lista pól. Dla pól typów złożonych istnieje możliwość zagnieżdżenia tej struktury. Przetworzenie zapytania w taki sposób pozwala na uniknięcie przesyłania niepotrzebnych pól zasobu, ponieważ żądane pola są jawnie określone w zapytaniu. Rozwiązuje to również problem $N+1$ zapytań, ponieważ możliwie jest wskazanie pola odnoszącego się do zasobu powiązanego, a także określenie, które pola tego zasobu mają zostać zwrócone. Możliwość przesyłania wielu żądań API w ramach jednego zapytania rozwiązuje również problem żądania wielu zasobów.

Ważnym elementem GraphQL jest definicja statycznie typowanego schematu API. W celu używania tego narzędzia konieczne jest utworzenie odpowiednich plików opisujących możliwe żądania, ich parametry, zwracane obiekty, odpowiednie typy. Język definicji schematu jest niezależny od języka programowania, w którym tworzone są aplikacje.

GraphQL implikuje inne spojrzenie na API aplikacji niż w przypadku RPC. Struktura zapytań nie odzwierciedla intuicyjnego dla programistów wywołania procedury lub metody. Wprowadza to dodatkową warstwę abstrakcji związaną z komunikacją między aplikacjami, którą trudno jest przedstawić w języku programowania. Skutkuje to potrzebą definiowania tej warstwy w osobnym, stworzonym specjalnie do tego celu języku. Powoduje to zwiększenie złożoności systemu i zwiększa nakład pracy potrzebny na obsługę komunikacji. Finalnie wymiana danych, tworzenie zapytań oraz operacje na otrzymanych w ich wyniku odpowiedziach odbywają się przy pomocy pewnego języka programowania, a więc konieczne jest odzwierciedlenie zapytań i odpowiedzi przy pomocy struktur tego języka. Takie rozwiązanie ma też swoje zalety, ponieważ pozwala na wykonywanie

wymiany danych pomiędzy aplikacjami w sposób bardziej niezależny od ich implementacji, użytych języków programowania i technologii. Ponadto sposób wykonywania zapytań jest bardzo intuicyjny, nie tylko dla programistów, przez co znajduje szersze zastosowanie.

Istnieją również propozycje rozwiązań przedstawionych w rozdziale trzecim problemów, które skupiają się na modelu RPC oraz REST. Przykładami są opublikowany przez Google poradnik designu API [goo] oraz utworzona przez Microsoft specyfikacja OData [mic]. Uwzględniają stosowanie pewnych mechanizmów ograniczających *underfetching* i *overfetching*, takich jak filtrowanie wyników danych, projekcja pól odpowiedzi, dołączanie do odpowiedzi powiązanych zasobów oraz paginacja. Propozycje te nie są powszechnie przyjętymi standardami i nie tłumaczą wpływu tych mechanizmów na tworzony system ani powodu ich wprowadzenia. Czytając je można wywnioskować, że autorzy mieli na uwadze problemy, które poruszane są w niniejszej pracy.

Kolejnym przykładem jest mechanizm Remote Monad [GSD⁺15], realizujący żądania przy pomocy typu monadycznego reprezentującego zapytanie do serwera. Wykonanie utworzonego zapytania odbywa się po wywołaniu funkcji monadycznej *send*. Pozwala to na agregację żądań RPC, dzięki czemu możliwe jest ograniczenie liczby wykonywanych zapytań. Implementacja tego mechanizmu jest możliwa tylko w językach funkcyjnych i wymaga rozumienia wielu zagadnień tego paradygmatu programowania. W połączeniu z mechanizmem monad comprehensions [mon] możliwe jest stworzenie implementacji komunikacji po stronie aplikacji klienta w taki sposób, że użycie Remote Monad jest transparentne dla programisty. Rozwiązanie to nie jest powszechnie stosowane ani rozwijane, lecz stanowi ciekawy przykład optymalizacji komunikacji.

Innym narzędziem wykorzystującym agregację żądań celu optymalizacji komunikacji jest BRMI [brm], stanowiące rozszerzenie Java RMI [PM01], mechanizmu języka Java służącego do tworzenia zdalnych wywołań metod, a więc koncepcyjnie komunikacji w modelu RPC. BRMI wspiera wykonywanie wielu wywołań po stronie klienta, które zostają zagregowane i wysłane na serwer dopiero do wywołaniu operacji *flush*.

Rozdział 5

Proponowane mechanizmy rozwiązujące problemy komunikacji między aplikacjami

Jak zostało opisane w rozdziale trzecim, problemy komunikacji między aplikacjami pojawiają się, gdy klient w ramach dostępnych w API żądań i parametrów nie jest w stanie w wyniku pojedynczego zapytania uzyskać dokładnie takiego rezultatu, jaki potrzebuje, przez co zmuszony jest do wykonania większej liczby zapytań bądź do otrzymania większej ilości danych niż jest to konieczne. W celu rozwiązania tych problemów możliwe jest zastosowanie dwóch podejść.

Pierwszym podejściem jest specjalizacja API, a więc tworzenie żądań w taki sposób, by były jak najbardziej dopasowane pod potrzeby konkretnego klienta. Im bardziej rezultat żądania odpowiada przypadkowi użycia po stronie klienta, tym większa jest wydajność systemu. Tego typu rozwiązanie jest często stosowane [New15], a jego zaletą jest prostota implementacji. Niestety zastosowanie tej taktyki zwiększa powiązanie API z aplikacją klienta, co powoduje powstanie systemu potencjalnie trudnego w utrzymaniu z uwagi na ciągłą potrzebę modyfikacji API do potrzeb klientów. Może to znacząco wydłużyć czas tworzenia systemu oraz zwiększyć bazę kodu aplikacji serwerowej. Tego typu rozwiązanie nie jest też możliwe do zastosowania w przypadku API przeznaczonego do użytku publicznego, gdy specyficzne potrzeby klientów nie są możliwe do ustalenia. Popularnym zastosowaniem tej taktyki jest architektura BFF (Backend for Frontends [New15]), polegająca na tworzeniu zupełnie osobnych API dla każdej z występujących w systemie aplikacji klienta, np. dla aplikacji mobilnej i przeglądarkowej.

Drugie podejście stosowane w celu rozwiązania problemów komunikacji polega na generalizacji, a więc skupieniu się przy projektowaniu API na tworzeniu żądań, które za pomocą parametrów mogą zostać dopasowane do specyficznej potrzeby klientów. To rozwiązanie może okazać się bardziej skomplikowane implementacyjnie. W zależności od użytych technologii istnieje możliwość wyodrębnienia implementacji tych mechanizmów do kodu infrastruktury, w taki sposób, by móc generycznie ich używać dla dowolnego żądania, dostarczając ewentualnie odpowiednią konfigurację. Przyjmując omówione podejście do implementacji, zastosowanie takiej taktyki może początkowo skomplikować tworzenie systemu, ale znacząco zmniejszy koszt jego utrzymania oraz rozbudowy. W wyniku tego rozwiązania klient otrzymuje większą kontrolę nad zadaniami wykonywanymi po stronie serwera, co zwiększa elastyczność API, ale może powodować problemy związane z bezpieczeństwem oraz w większym stopniu obciążać aplikację serwerową.

Poniżej znajduje się opis proponowanych w niniejszej pracy mechanizmów opartych na przedstawionych taktykach, które zostały zaimplementowane oraz poddane testom.

5.1 Żądanie dostosowane do konkretnego przypadku użycia.

W tym rozwiązaniu w API tworzone jest żądanie, które bez przekazywania dodatkowych parametrów pozwoli zrealizować wykonywany przez klienta przypadek użycia za pomocą pojedynczego zapytania i jego rezultat będzie idealnie dopasowany do potrzeb klienta. Rozwiązanie to może być stosowane w praktyce, jednak z uwagi na stwarzaną przez nie trudność utrzymania systemu, często wybierane jest użycie jego mniej restrykcyjnej formy, gdzie żądanie jest tylko częściowo dopasowane do potrzeb klienta. Takie rozwiązanie stanowi również punkt odniesienia dla innych proponowanych mechanizmów.

Potencjalnie jego zastosowanie pozwala rozwiązać wszystkie przedstawione w rozdziale trzecim problemy komunikacji.

5.2 Projekcja zasobu przy pomocy parametru “fields”.

Mechanizm ten polega na przekazaniu jako parametr żądania listy pól zasobu, które mają zostać uzyskane w odpowiedzi. Dzięki temu klient może dostosować odpowiedź do potrzeb konkretnego przypadku użycia API i ograniczyć w ten sposób rozmiar przesyłanych danych, a także zmniejszyć koszt przetwarzania odpowiedzi po stronie klienta.

Mechanizm ten pozwala na rozwiązanie problemu przesyłania niepotrzebnych pól zasobu opisanego w rozdziale drugim.

5.3 Połączenie zasobu powiązanego przy pomocy parametru “include”.

W tym mechanizmie, jako parametr żądania przekazywana jest lista pól zasobu, które są identyfikatorami innych, powiązanych zasobów. Powiązane zasoby mają zostać dołączone do odpowiedzi, eliminując w ten sposób potrzebę wykonywania przez klienta dodatkowych zapytań w celu ich uzyskania. Użycie tego mechanizmu wymaga dostarczenia odpowiedniej konfiguracji w aplikacji serwerowej, określającej w jaki sposób uzyskać powiązany zasób przy pomocy jego identyfikatora. Zwiększa to nieznacznie koszt zastosowania tego mechanizmu.

Mechanizm ten pozwala na rozwiązanie problemu N+1 zapytań opisanego w rozdziale drugim.

5.4 Agregacja zapytań

To rozwiązanie polega na ograniczeniu liczby wykonywanych przez klienta zapytań poprzez agregację żądań w ramach jednego zapytania. W ten sposób możliwe jest poprawienie wydajności systemu bez potrzeby komplikowania projektu API oraz kodu aplikacji, co ma miejsce przy zastosowaniu którejkolwiek z opisanych na początku tego rozdziału taktów. Działanie tego mechanizmu może być w całości wyodrębnione do kodu infrastruktury. Wadą rozwiązania jest komplikacja wysyłania żądań po stronie aplikacji klienta.

Zastosowanie agregacji zapytań pozwala na rozwiązanie problemu żądania wielu zasobów, opisanego w rozdziale drugim.

Przy pomocy agregacji jest też możliwe częściowe rozwiązanie problemu N+1 zapytań, ponieważ żądania o powiązane zasoby można zagregować do jednego zapytania, ostatecznie realizując cały przypadek użycia przy pomocy jedynie dwóch zapytań. W ogólności jest to rozwiązanie mniej wydajne niż parametr “include”, jednak w szczególnym przypadku może pozwalać na osiągnięcie większej wydajności. Przy założeniu, że na liście zasobów otrzymanych w wyniku pierwszego zapytania w polu identyfikującym zasób powiązany występują powtórzenia, to może okazać się, że w wyniku zastosowania parametru “include” połączony zasób jest przesyłany w odpowiedzi wielokrotnie. Można tego uniknąć, gdy zastosuje się agregację zapytań, ponieważ klient może przed wysłaniem drugiego zapytania usunąć powtórzone żądania o ten sam zasób. Zastosowanie agregacji może jednak skomplikować kod aplikacji klienta w stosunku do zastosowania parametru “include” przez konieczność obsłużenia większej liczby żądań oraz ręcznego połączenia zasobów powiązanych już po stronie klienta.

5.5 Jednoczesne zastosowanie ww. mechanizmów

Przedstawione mechanizmy w wielu przypadkach mogą zostać zastosowane jednocześnie dla danego przypadku użycia, w zależności od tego, jakie problemy komunikacyjne się w nim pojawiają.

Stosując parametr “fields” razem z parametrem “include” pojawia się możliwość projekcji pól również zasobów powiązanych. Niestety używanie tych dwóch parametrów razem jest problematyczne, gdyż klient zmuszony jest przedstawić strukturę drzewiastą hierarchii zasobów w płaskiej strukturze listy pól. Dodatkowo klient musi zachować spójność między parametrami, nie jest na przykład w stanie dokonać projekcji pól powiązanego zasobu, który nie został połączony oraz połączyć zasobu, którego pole zostało usunięte z odpowiedzi w wyniku projekcji. Sprawia to, że użycie tych mechanizmów razem znacząco się komplikuje, a także zwiększa się ryzyko błędów po stronie klienta.

Istnieje możliwość zastosowania agregacji zapytań zamiast parametru “include”, co pozwala na uniknięcie komplikacji wynikających ze stosowania razem parametrów “fields” i “include”, ale ma wady opisane powyżej w punkcie 5.4.

5.6 Optymalizacja na poziomie logiki biznesowej i warstwy dostępu do danych

Niektóre z zastosowanym mechanizmów mogą potencjalnie pozwolić na dalszą optymalizację obsługi żądań. W niniejszej pracy skupiono się jedynie na optymalizacji na poziomie komunikacji, co oznacza, że na poziomie logiki biznesowej oraz warstwie dostępu do danych nie zostały wprowadzone żadne zmiany. W celu optymalizacji na innych warstwach konieczne jest silne powiązanie ich z warstwą komunikacji oraz czasochłonna implementacja mechanizmów, często niemożliwa do wykonania w sposób generyczny. Proponowane w pracy rozwiązania mają być możliwe do zastosowania niezależnie od specyficznej dla aplikacji logiki biznesowej oraz zastosowanego sposobu dostępu do danych.

Rozdział 6

Wybrane przypadki użycia i scenariusze testowe

Na potrzeby przeprowadzenia analizy i testów wydajnościowych zostały wybrane przypadki użycia API. Zostały one dobrane w taki sposób, by uwidocznic przedstawione w pracy problemy komunikacji. Dla poszczególnych przypadków użycia zostały stworzone scenariusze testowe, w których zostały zastosowane proponowane mechanizmy komunikacji. Poniżej znajduje się lista proponowanych przypadków użycia oraz scenariuszy testowych:

6.1 Przypadek testowy 1: Przesyłanie niepotrzebnych pól zasobu

W tym przypadku użytkownik pobiera przy pomocy API listę dostępnych ebook'ów. Potrzebne mu są jednak jedynie pola: identyfikator ebooka oraz tytuł ebooka. Poza tymi polami zasób zawiera również inne pola, takie jak: identyfikator właściciela, podsumowanie, URL okładki czy statystyki. W przypadku pobrania wszystkich tych pól pojawia się więc problem przesyłania niepotrzebnych pól zasobu. W ramach przypadku testowego zostały przeprowadzone następujące scenariusze testowe:

- Pobranie listy ebook'ów bez zastosowania żadnego mechanizmu
- Pobranie listy ebook'ów z zastosowaniem parametru „*fields*”
- Pobranie listy książek przy pomocy żądania dostosowanego do przypadku użycia

6.2 Przypadek testowy 2: Problem N+1 zapytań

W ramach tego przypadku testowego użytkownik chce pobrać listę dostępnych w systemie promocji. Jednym z pól zasobu promocji jest identyfikator kursu, którego ona dotyczy. Użytkownik dla każdej z otrzymanych promocji chce uzyskać również odpowiadający jej zasób kursu. Bez zastosowania żadnego mechanizmu pojawia się problem N+1 zapytań.

Dla opisanego przypadku przeprowadzono następujące scenariusze testowe:

- Pobranie listy promocji wraz z kursami bez zastosowania żadnego mechanizmu
- Pobranie listy promocji wraz z kursami z zastosowaniem parametru „*include*”
- Pobranie listy promocji wraz z kursami z zastosowaniem agregacji zapytań
- Pobranie listy promocji wraz z kursami z zastosowaniem agregacja zapytań i optymalizacji powtórzeń
- Pobranie listy promocji wraz z kursami przy pomocy żądania dostosowanego do przypadku użycia

6.3 Przypadek testowy 3: Żądanie wielu zasobów

W następującym przypadku testowym użytkownik potrzebuje kilku niezależnych zasobów: listy promocji, czterech różnych kursów, listy sekcji jednego z kursów oraz listy ebooków. W tym celu, jeśli nie zostanie zastosowany żaden dodatkowy mechanizm, musi on wykonać serię osobnych żądań o poszczególne zasoby.

Dla tego przypadku zostały przeprowadzone następujące scenariusze testowe:

- Pobranie potrzebnych zasobów bez zastosowania żadnego mechanizmu
- Pobranie potrzebnych zasobów z zastosowaniem agregacji zapytań
- Pobranie potrzebnych zasobów przy pomocy żądania dostosowanego do przypadku użycia

Rozdział 7

Wybrane technologie

Na potrzeby implementacji proponowanych w pracy mechanizmów wybrano język programowania Kotlin [SB17] działający na maszynie wirtualnej Javy (JVM) [Ven98]. Kotlin jest obiektywnym językiem statycznie typowanym stworzonym z myślą o tworzeniu aplikacji biznesowych. Kod pisany w tym języku jest w pełni kompatybilny z kodem napisanym w Javie. Składania tego języka jest jednak bardziej zwięzła. Natywnie wspierane jest wiele dobrych wzorców programowania znajdujących zastosowanie szczególnie w kontekście aplikacji biznesowych.

Testowana w ramach pracy aplikacja zaimplementowana została w Kotlinie, co wpłynęło na decyzję o wykorzystaniu tej technologii również do implementacji innych elementów pracy. Został on również wybrany z uwagi na preferencję autora pracy.

Do implementacji komunikacji przy pomocy protokołu HTTP została wykorzystana biblioteka ktor [kto]. Jest to biblioteka napisana w języku Kotlin i służy do obsługi komunikacji między aplikacjami. Pozwala na definiowanie API aplikacji serwerowej w sposób deklaratywny oraz asynchroniczną obsługę żądań przy pomocy dostępnego w języku Kotlin mechanizmu współprogramów [Mar80]. Proponowane w pracy mechanizmy komunikacji zostały zaimplementowane jako rozszerzenie tej biblioteki.

W celu przeprowadzenia testów wydajnościowych zostało wykorzystane narzędzie Locust [loc]. Pozwala ono na przeprowadzenie testów obciążeniowych aplikacji. Narzędzie to wyróżnia możliwość definiowania scenariuszy testowych za pomocą kodu, bez konieczności ich konfiguracji przy pomocy interfejsu użytkownika lub plików konfiguracyjnych. Locust jest napisany w języku Python i domyślnie w tym języku pisane są scenariusze testowe. Twórcy umożliwili jednak uruchomienie aplikacji w trybie serwera i klientów, w którym klienci wykonujące scenariusze testowe wysyłają do serwera wyniki poszczególnych wywołań przy pomocy API. Dzięki temu scenariusze testowe mogą zostać opisane w języku Kotlin.

Do uruchomienia testów, agregacji wyników i wygenerowania wykresów konieczne było napisanie skryptów automatyzujących te zadania. W tym celu został użyty język powłoki systemowej bash [NR05] oraz skryptowa wersja języka Kotlin. Do generacji grafów została użyta biblioteka ggplot2 [Wic16].

Rozdział 8

Implementacja

8.1 Biblioteka kucik-api

W ramach niniejszej pracy została stworzona biblioteka do języka Kotlin o nazwie kuick-api, która wspiera tworzenie API w modelu RPC. Narzędzie to zostało stworzone jako rozszerzenie biblioteki ktor. Dostarcza ono generyczne metody rozbudowujące zapewniony przez bibliotekę ktor DSL [VDKV00] służący do definicji API aplikacji serwerowej. Pozwala w sposób deklaratywny stworzyć API w modelu RPC dla wskazanej klasy. Poniżej znajduje się przykład użycia biblioteki:

```
1 application.routing {  
2     route("/rpc") {  
3         rpcRoute<ResourceApi>(injector)  
4     }  
5 }
```

LISTING 8.1: Deklaracja API RPC dla klasy ResourceApi

W pierwszej linii otwierany jest blok deklaracji API dla aplikacji. W linii drugiej otwierany jest blok ścieżki `"/rpc"`, co oznacza, że wszystkie żądania API zdefiniowane w tym bloku będą posiadały taki właśnie prefiks w adresie URL. W trzeciej linii wywoływana jest metoda biblioteki kuick-api, która utworzy żądania w stylu RPC dla poszczególnych metod klasy ResourceApi.

Przekazana do metody `rpcRoute` klasa jest dynamicznie analizowana przy pomocy mechanizmu refleksji. Dla każdej z zadeklarowanych w ramach klasy metod o dostępie publicznym generowana jest odpowiednia obsługa zapytania HTTP. Adres URL zapytania składa się ze zdefiniowanego prefiksu, nazwy klasy oraz nazwy metody. W ciele zapytania przesyłane są parametry metody o odpowiednich typach. Odpowiedzią na zapytanie w przypadku sukcesu będzie instancja klasy zwracanej przez metodę. Dla przykładu, dla następującej klasy ResourceAPI

```
1 data class Resource(  
2     val id: String? = null,  
3     val field1: String,  
4     val field2: Int,  
5     val otherResource: String? = null  
6 )  
7  
8 class ResourceApi {  
9     fun getAll(): List<Resource> = /*implementacja*/  
10    fun addResource(resource: Resource) : Boolean = /*implementacja*/  
11 }
```

LISTING 8.2: Przykładowa klasa ResourceApi

zostaną utworzone następujące żądania API:

1. `POST /rpc/ResourceApi/getAll`, które nie przyjmuje parametrów w ciele metody, a zwraca listę zasobów Resource,
2. `POST /rpc/ResourceApi/addResource`, które w ciele metody przyjmuje tablicę zawierającą instancję klasy Resource, a zwraca nowo utworzony zasób Resource.

W ramach obsługi zapytania przekazywane parametry są poddawane deserializacji, a następnie wywoływana jest odpowiednia metoda, której wynik jest serializowany i odsyłany jako wynik zapytania. Do serializacji oraz deserializacji użyta została biblioteka Jackson [jac].

Biblioteka obsługuje również mechanizm dodatkowych parametrów, co pozwala na przekazanie do wywołania metody parametrów, nawet jeśli nie zostały one zamieszczone w ciele zapytania. Zastosowanie mechanizmu pokazuje poniższy przykład:

```

1 application.routing {
2     // withParameter("session") { getSessionFromRequest() }
3     route("/rpc") {
4         // withParameter("session") { getSessionFromRequest() }
5         rpcRoute<ResourceApi>(injector) {
6             withParameter("session") { getSessionFromRequest() }
7         }
8     }
9 }

```

LISTING 8.3: Użycie mechanizmu dodatkowego parametru

W linii 6 znajduje się konfiguracja, pozwalająca na dodanie do wywołania metody parametru „session”, który nie jest przekazywany w ciele zapytania, lecz otrzymywany w inny sposób. Konfiguracja ta może zostać zastosowana zarówno na poziomie deklaracji API dla klasy (jak w powyższym przykładzie), jak i na poziomie bloku ścieżki (linia 4) lub deklaracji API całej aplikacji (linia 2). Wówczas jej działanie obejmuje wszystkie zdefiniowane przy użyciu biblioteki żądania RPC, w ramach bloku, w którym została umieszczona.

W ramach biblioteki zostały zdefiniowane typy pozwalające na komunikację klienta z API. Należą do nich generyczne klasy reprezentujące zapytanie klienta i odpowiedź serwera:

```

1 data class Request<T>{
2     val path: String,
3     val queryParameters: Map<String, List<String>>,
4     val body: String
5 }
6
7 sealed class Response<T> {
8     data class Success<T>{
9         val result: T,
10        val status: String,
11        val message: String? = null
12    } : Response<T>()
13
14    data class Failure<T>{
15        val status: String,
16        val message: String? = null
17    } : Response<T>()
18 }

```

LISTING 8.4: Klasy Request i Response

Została również zdefiniowana klasa abstrakcyjna Device, zawierająca metodę służącą do wysyłania zapytań:

```

1 interface Device {
2     fun call(request: Request<*>): HttpResponse
3 }
4
5 inline fun <reified A : Any> Device.send(request: Request<A>): Response<A>
6     = call(request).toResponse<A>()

```

LISTING 8.5: Metoda send klasy Device

Takie rozwiązanie pozwala na szybkie tworzenie kodu klienta API. Dla klas z aplikacji serwerowej, dla których zostały stworzone żądania RPC, konieczne jest utworzenie kodu klienta zwracającego odpowiednie obiekty klasy Request. Poniżej znajduje się przykład takiego kodu:

```

1 class ResourceApiClient {
2     fun getAll(): Request<List<Resource>> = Request(
3         path = "/rpc/ResourceApi/getAll",
4         queryParameters = emptyMap(),
5         body = emptyJson()
6     )
7
8     fun addResource(resource: Resource): Request<Boolean> = Request(
9         path = "/rpc/ResourceApi/getAll",
10        queryParameters = emptyMap(),
11        body = jsonArrayOf(resource.toJson())
12    )
13 }

```

LISTING 8.6: Przykład kodu klienta dla klasy ResourceApi

Zdefiniowane nazwy, ich parametry oraz zwracane typy odpowiadają tym z bazowej klasy ResourceApi. W obecnym stanie biblioteki konieczne jest napisanie tego kodu ręcznie, jednak przy pomocy refleksji i mechanizmu generacji kodu możliwy jest rozwój biblioteki w taki sposób, by kod ten generować automatycznie dla zdefiniowanego API RPC. W aplikacji klienta konieczne jest wówczas jedynie rozszerzenie klasy Device, dla której należy zaimplementować metody specyficzne dla użytej technologii do wykonywania zapytań HTTP. Poniżej znajduje się przykład wykonania zapytania RPC w aplikacji klienta:

```

1 val resourceApi = ResourceApiClient()
2
3 device.send(resourceApi.addResource(someResource))
4
5 val resources : Response<List<Resource>>
6     = device.send(resourceApi.getAll())

```

LISTING 8.7: Przykład zapytania RPC w aplikacji klienta

W linii 3 znajduje się wywołanie zapytania pobierającego zasób, a w linii 6 wywołanie zapytania pobierającego listę zasobów.

8.2 Implementacja mechanizmu projekcji zasobu

W ramach biblioteki quick-api został zaimplementowany mechanizm projekcji zasobu. Aktywacja mechanizmu odbywa się poprzez konfigurację w bloku definicji API. Przedstawia to poniższy przykład:

```

1 application.routing {
2     route("/rpc") {
3         withFieldsParameter()
4         rpcRoute<ResourceApi>(injector)
5     }
6 }

```

LISTING 8.8: Konfiguracja projekcji zasobu

Metoda wywołana w linii 3 powoduje dodanie do obsługi zapytania procedury odczytującej z niego parametr „\$fields”, umieszczony w URL. Poprzedzony jest on znakiem „\$” w celu uniknięcia ewentualnej kolizji nazw. Podobnie jak dla metody *withParameter*, jej działanie obejmuje wszystkie zdefiniowane przy użyciu biblioteki żądania RPC, w ramach bloku, w którym została umieszczona.

Przed wysłaniem odpowiedzi do klienta, odczytany parametr jest analizowany i użyty w celu ograniczenia przesyłanych danych do odpowiednich pól zasobów. Jest to wykonywane na etapie serializacji. Wartością parametru jest lista nazw pól zasobu. W przypadku, gdy przesyłany jest również zasób powiązany, pola dla tego zasobu określane są w formie ścieżki, w której elementy oddzielone są kropką, np.

POST /rpc/ResourceApi/getAll?\$fields=[„id”, „field1”, „otherResource.id”, „otherResource.field2”].

Biblioteka wspiera wykonywanie zapytań o zasoby poddane projekcji, w taki sposób, by zachować zgodność typów. Dla bazowego zapytania, które jak opisano wcześniej, mogłoby zostać wygenerowane przez kod biblioteki, użytkownik może użyć metody *modify*, która dodaje do niego przekazane parametry i zmienia typ odpowiedzi. Obrazuje to poniższy przykład:

```
1 data class ModifiedResource(  
2     val field1: String  
3 )  
4  
5 fun ResourceApi.getAllModified() = resourceApi.getAll()  
6     .modify<List<ModifiedResource>>("\$fields" to listOf("field1"))  
7  
8 val resources : Response<List<ModifiedResource>> = device.send(  
9     resourceApi.getAllModified()  
10 )
```

LISTING 8.9: Użycie projekcji zasobu przy wykonywaniu zapytania

W linii 5 zdefiniowana została nowa metoda klasy *ResourceApi*, która modyfikuje bazowe żądanie pobierające listę zasobów. Została ona użyta w linii 9.

8.3 Implementacja mechanizmu połączenia zasobu powiązanego

Konfiguracja mechanizmu połączenia zasobu powiązanego odbywa się w podobny sposób co projekcji zasobu, wymagane jest jednak przekazanie dodatkowych parametrów. Domyślnie w polu zasobu powiązanego przesyłany jest identyfikator tego zasobu. W wyniku działania opisywanego rozwiązania, przy pomocy identyfikatora jest uzyskiwany odpowiedni zasób, po czym wstawiany jest on jako wartość pola w odpowiedzi, w miejsce identyfikatora. W konfiguracji, w aplikacji serwerowej, konieczne jest określenie sposobu, w jaki jest uzyskiwany zasób na podstawie identyfikatora. Przedstawia to poniższy przykład:

```
1 val otherResourceApi = injector.getInstance(OtherResourceApi::class.java)  
2  
3 application.routing {  
4     route("/rpc") {  
5         withIncludeParameter(  
6             Resource::otherResource to { id -> otherResourceApi.getOne(id)}  
7         )  
8         rpcRoute<ResourceApi>(injector)  
9     }  
10 }
```

LISTING 8.10: Konfiguracja połączenia zasobu powiązanego

W linii 5 wywołano metodę powodującą dodanie do obsługi zapytania procedury odczytującej z niego parametr „*\$include*”, umieszczony w URL. Jej działanie obejmuje wszystkie zdefiniowane przy użyciu biblioteki żądania RPC, w ramach bloku, w którym została umieszczona. W linii 6 przekazano parametr konfiguracji, mówiący że identyfikator z pola „*otherResource*” zasobu *Resource* ma być mapowany do odpowiedniego zasobu poprzez wywołanie metody *getOne* klasy *OtherResourceApi*.

Parametr „*include*” jest analizowany przed wysłaniem odpowiedzi do klienta. Jego format jest podobny do formatu parametru „*fields*”, a więc zawiera listę pól zasobu oraz zasobów powiązanych, które mają zostać rozwinięte i dołączone do odpowiedzi. Na etapie serializacji wywoływane są odpowiednie metody przekazane w konfiguracji, po czym ich wynik również jest serializowany i dołączany do odpowiedzi w miejsce identyfikatora. Obsługa połączenia zasobu powiązanego ma miejsce przed projekcją zasobu, w celu obsługi projekcji również dla zasobów powiązanych.

Wykonywanie zapytań o zasoby wraz z połączonymi zasobami powiązanymi odbywa się w analogiczny sposób jak przy projekcji zasobu. Użytkownik może zmodyfikować bazowe zapytanie poprzez dodanie do niego parametru i zmienienie typu odpowiedzi. Widać to na poniższym przykładzie:

```

1 data class ModifiedResource(
2     val id: String,
3     /* inne pola */
4     val otherResource: OtherResource?
5     /* zamiast
6     val otherResource: String?
7     */
8 )
9
10 fun ResourceApi.getAllModified() = resourceApi.getAll()
11     .modify<List<ModifiedResource>>(
12         "\$include" to listOf("otherResource")
13     )
14
15 val resources : Response<List<ModifiedResource>> = device.send(
16     resourceApi.getAllModified()
17 )

```

LISTING 8.11: Użycie połączenia zasobu powiązanego przy wykonywaniu zapytania

8.4 Implementacja mechanizmu agregacji zapytań

W bibliotece został również zaimplementowany mechanizm agregacji zapytań. Zapytania używające tego mechanizmu są wysyłane w ramach specjalnego żądania API. Aby zdefiniować to żądanie w API serwera i tym samym włączyć obsługę agregacji zapytań, konieczna jest następująca konfiguracja:

```

1 application.routing {
2     route("/rpc") {
3         rpcRoute<ResourceApi>(injector)
4         aggregationRoute(injector)
5     }
6 }

```

LISTING 8.12: Konfiguracja agregacji zapytań

W linii 4 została wywołana metoda, która do definicji API dodaje żądanie *POST /rpc/aggregation*. W ciele zapytania wysyłana jest lista obiektów klasy *Request*, reprezentująca odpowiednie żądania API. Dla z obiektów wykonywana jest odpowiednia obsługa, tak jakby zapytanie zostało wysłane bezpośrednio na konkretny adres API. Obsługiwane są jedynie żądania odpowiadające zdefiniowanym przy pomocy biblioteki żądaniom RPC. Wynikiem zapytania jest lista obiektów klasy *Response*. Działanie agregacji symuluje więc wysyłanie zapytań osobno na poszczególne adresy, lecz zarówno żądania i odpowiedzi są tu połączone w listę i przesyłane w ramach jednego zapytania.

Dokonana implementacja zakłada synchroniczną obsługę żądań. Jest to decyzja podjęta na potrzebę spełnienia założeń wykonanych w ramach pracy testów. Możliwe do zaimplementowania jest jednak asynchroniczne przetwarzanie żądań, a także dokonanie dalszych optymalizacji takich np. jak obsługa powtórzonych żądań tylko jeden raz.

Ważnym elementem biblioteki jest dostarczenie mechanizmów pozwalających wykonywać zregrowane zapytanie w możliwie transparentny sposób i przy zachowaniu wsparcia dla deserializacji do odpowiednich typów. Metody biblioteki, których odpowiedzialnością jest obsługa komunikacji po stronie klienta, mają charakter generyczny w celu zapewnienia funkcjonalności deserializacji do zwracanego przez zdalne wywołanie typu danych. Przy agregacji zapytań konieczne jest, by metoda zwracała wiele wyników, potencjalnie różnych typów. W języku Kotlin nie istnieje możliwość implementacji takiej generycznej metody, działającej dla dowolnej liczby zmiennych. Z tego powodu implementacja została dokonana w sposób statyczny, przy pomocy mechanizmu generacji kodu. Wygenerowano odpowiednie klasy pozwalające na zwrócenie wielu wyników różnych typów:

```

1 data class Tuple1<out A>(val _1: A)
2 data class Tuple2<out A, out B>(val _1: A, val _2: B)
3 data class Tuple3<out A, out B, out C>(val _1: A, val _2: B, val _3: C)
4 // ...

```

LISTING 8.13: Definicja klas TupleX

Został również wygenerowany obiekt Tuple, upraszczający tworzenie obiektów poszczególnych klas w zależności od liczby parametrów.

```

1 object Tuple {
2     operator fun <A> invoke(_1: A): Tuple1<A>
3         = Tuple1(_1)
4     operator fun <A, B> invoke(_1: A, _2: B): Tuple2<A, B>
5         = Tuple2(_1, _2)
6     operator fun <A, B, C> invoke(_1: A, _2: B, _3: C): Tuple3<A, B, C>
7         = Tuple3(_1, _2, _3)
8     // ...
9 }

```

LISTING 8.14: Definicja obiektu Tuple

Zdefiniowano dekorator interfejsu Device, pozwalający na wykonywanie zapytań zagregowanych.

```

1 abstract class AggregatingDevice<R>(device: Device<R>) : Device<R> by device {
2     abstract fun aggregatedCall(packet: List<Request<*>>): R
3 }

```

LISTING 8.15: Dekorator AggregatingDevice

Dla klasy AggregatingDevice zostały wygenerowane wersje metody send dla poszczególnych liczb żądań, pozwalające na wysyłanie zapytań zagregowanych. Poniżej znajduje się przykład wersji metody send dla agregacji dwóch żądań:

```

1 inline fun <reified A : Any, reified B : Any, R> AggregatingDevice<R>.send(
2     _1: Request<A>,
3     _2: Request<B>
4 ): Tuple2<Response<A>, Response<B>> {
5     val result = aggregatedCall(listOf(_1, _2))
6     val responsesJson = Json.jsonParser
7         .parse(result.getBody())
8         .asJsonArray
9     return Tuple2(
10         responsesJson[0].toResponse<A>(),
11         responsesJson[1].toResponse<B>()
12     )
13 }

```

LISTING 8.16: Metoda send dla agregacji zapytań

Implementacja dla większej liczby parametrów wygląda analogicznie. W ramach biblioteki wygenerowano metody obsługujące do 26 parametrów, jednak ta liczba może zostać zwiększona.

Zastosowanie takiej implementacji pozwala na transparentne wywoływanie agregacji zapytań po stronie aplikacji klienta. Poniżej znajduje się przykład obsługi zagregowanego zapytania przez klienta:

```

1 val (resource1, resource2, otherResource1, otherResource2) = device.send(
2     resourceApi.getOne("resource-id-1"),
3     resourceApi.getOne("resource-id-2"),
4     otherResourceApi.getOne("other-resource-id-1"),
5     otherResourceApi.getOne("other-resource-id-2")
6 )

```

LISTING 8.17: Wywołanie zagregowanego zapytania

gdzie resource1, resource2, otherResource1 i otherResource2 to wyniki odpowiednich zapytań, poddane deserializacji do odpowiednich typów, np. resource1 do typu Response<Resource>.

8.5 Implementacja aplikacji testowej

Do testów wykonanych w ramach pracy została wykorzystana hiszpańska platforma e-learningowa GoKoan. Aplikacja serwerowa została zmodyfikowana na potrzeby testów, tak by wykorzystywać bibliotekę kuick-api. Konfiguracja jest analogiczna do przedstawionych w tym rozdziale przykładów.


```
1 val courseApi = injector.getInstance(CoursesApi::class.java)
2
3 //...
4
5 application.routing {
6     //...
7     route("/rpc") {
8         withFieldsParameter()
9         withIncludeParameter(
10             PromotionDetails::courseId to { id ->
11                 courseApi.getCourseById(id)
12             }
13         )
14
15         rpcRoute<LibraryApi>(injector)
16         rpcRoute<CoursesApi>(injector)
17         rpcRoute<PromotionsApi>(injector)
18         rpcRoute<CourseSectionsApi>(injector)
19
20         aggregationRoute(injector)
21
22         withParameter(
23             "session",
24             ifNull = { call.respond(HttpStatusCode.Forbidden) }) {
25             call.sessions.get<Session>()
26         }
27     }
28     //...
29 }
```

LISTING 8.18: Konfiguracja aplikacji testowej

W liniach 22-26 została dodana dodatkowego parametru „*session*”, używanego do autoryzacji zapytań.

Zostały również osobno zaimplementowane żądania API dostosowane do przypadku użycia.

```
1 application.routing {
2     //...
3     post("/case1") { /*implementacja*/ }
4     post("/case2") { /*implementacja*/ }
5     post("/case3") { /*implementacja*/ }
6     //...
7 }
```

LISTING 8.19: Żądania dostosowane do przypadku użycia

Rozdział 9

Testy

Testy wydajnościowe zostały wykonane w środowisku sieciowym, w którym komunikacja odbywała się przez Internet. Testowana aplikacja oraz aplikacja testująca zostały uruchomione na maszynach znajdujących się we Frankfurcie, w ramach platformy Google Cloud Platform [gcp]. Aplikacje zostały uruchomione na maszynach wirtualnych dostępnych w ramach usługi Compute Engine. Obydwie maszyny posiadają następujące parametry:

vCPUs: 1

Pamięć: 3,75 GB

Baza danych, z której korzysta aplikacja testowa, została uruchomiona na maszynie wirtualnej w ramach oferowanej przez Google Cloud usługi SQL:

vCPUs: 1

Pamięć: 3,75 GB

Rozmiar dysku: 10 GB

Typ dysku: SSD

System zarządzania bazą danych: PostgreSQL 9.6

Testy zostały uruchomione dla każdego przypadku użycia osobno w celu uniknięcia obciążenia testowanej aplikacji obsługą zapytań z pozostałych testów. W ramach każdego testu dany scenariusz testowy był powtarzany natychmiastowo po jego wykonaniu przez zadaną ilość czasu. Dla każdego wywołania badany był czas oraz rozmiar odpowiedzi. Czas odpowiedzi oznacza sumaryczny czas obsługi poszczególnych zapytań, a więc zakłada synchroniczny model przetwarzania po stronie klienta. Z tego powodu w przypadku agregacji zapytań implementacja tego mechanizmu po stronie serwera również przebiega w sposób synchroniczny. Rozmiar odpowiedzi uwzględniał nagłówki i ciało odpowiedzi HTTP oraz wyestymowany rozmiar nagłówków niższych warstw sieciowych. Przyjęto wartość 66B, zakładając że komunikacja będzie się odbywać przy pomocy protokołów IPv4 oraz Ethernet. Na podstawie tych danych możliwe było obliczenie takich parametrów jak średni czas odpowiedzi, średni rozmiar odpowiedzi oraz średnia liczba obsłużonych zapytań na sekundę. Testy były uruchamiane dla rosnącej wykładniczo liczby użytkowników w celu uzyskania większej dynamiki wzrostu tej liczby. Liczba użytkowników oznacza w praktyce liczbę wątków równoległe wykonujących zapytania do aplikacji testowanej. Dla każdej liczby użytkowników został wykonany osobny test trwający zadaną ilość czasu. Poniżej znajduje się opis parametrów przyjętych w przeprowadzonych testach:

Czas trwania pojedynczego testu: 1 minuta

Testowane liczby użytkowników: 1, 2, 4, 8, 16, 32, 64

Liczba testowanych przypadków użycia: 11

Ostatecznie wykonano więc 77 testów, z których każdy trwał 1 minutę. Wyniki testów zostały zebrane w formacie .csv w celu poddania ich dalszej analizie.

W ramach pracy poza wydajnością oceniany jest również wpływ zastosowania danego mechanizmu na szybkość tworzenia i łatwość utrzymania systemu. Niestety tego typu parametry trudno jest poddać testom. W związku z tym zdecydowano się na ocenę tych parametrów na podstawie analizy oraz subiektywnej oceny i doświadczenia autora pracy.

Rozdział 10

Wyniki

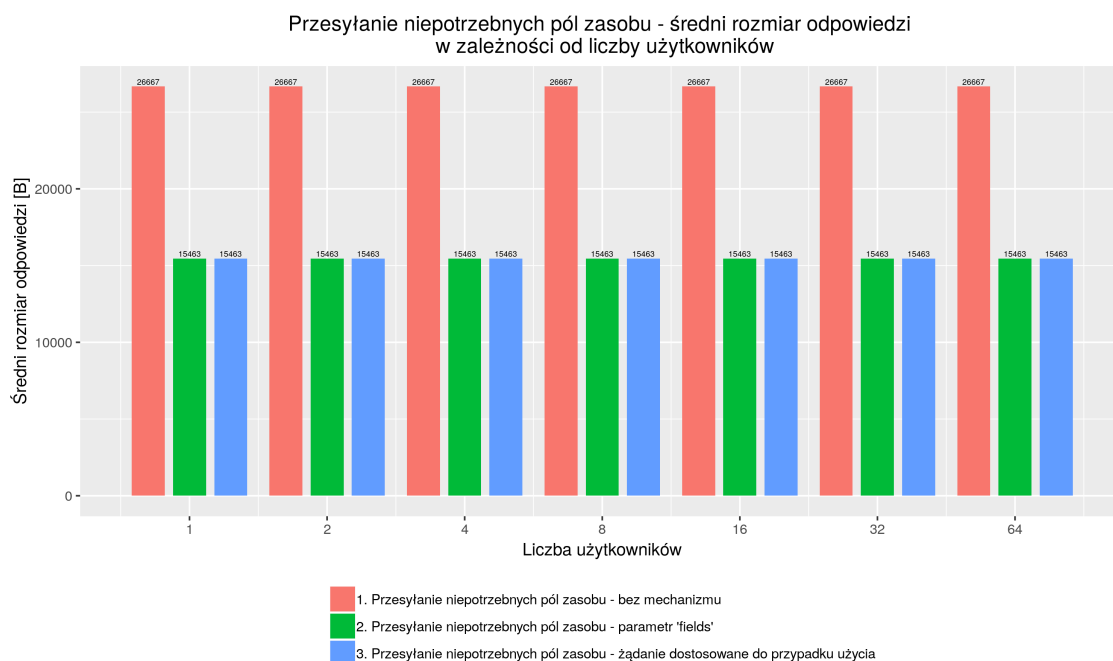
W niniejszym rozdziale zostały przedstawione oraz opisane wyniki wykonanych testów wydajnościowych, oraz analiza pod względem kosztów tworzenia i utrzymania systemów dla poszczególnych mechanizmów zaproponowanych w pracy.

10.1 Analiza testów wydajnościowych

Poniżej znajduje się analiza wyników testów wydajnościowych przeprowadzonych dla przypadków użycia i scenariuszy testowych omówionych w rozdziale 6.

10.1.1 Przesyłanie niepotrzebnych pól zasobu

Kluczowym parametrem dla tego problemu jest rozmiar uzyskiwanej w wyniku zapytania odpowiedzi. Poniżej znajduje się rysunek przedstawiający średni rozmiar odpowiedzi w zależności od liczby użytkowników dla poszczególnych wariantów zastosowanych mechanizmów:

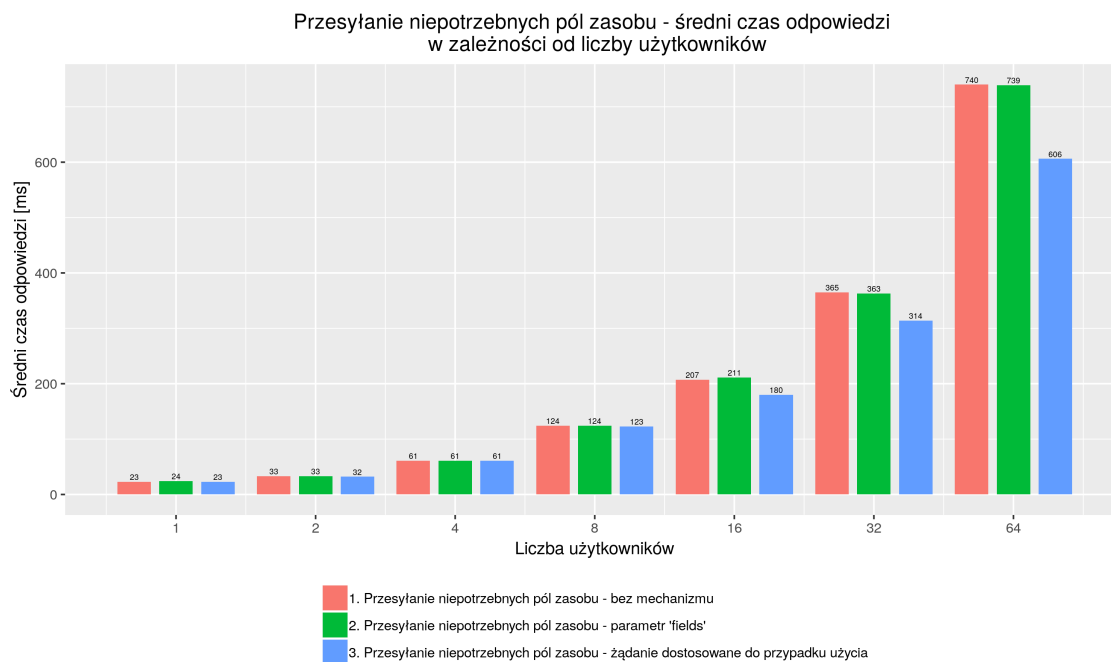


RYСУNEK 10.1: Przesyłanie niepotrzebnych pól zasobu – średni rozmiar odpowiedzi w zależności od liczby użytkowników

Średni rozmiar odpowiedzi nie jest zależny od liczby użytkowników. Na rysunku widać jednak, że zmienia się on w zależności od zastosowanego mechanizmu. Wynosi on 26733B przy braku optymalizacji i 15529B dla zastosowania projekcji zasobu oraz dla żądania dostosowanego do przypadku użycia. Uzyskany w wyniku zastosowania proponowanych mechanizmów rozmiar odpowiedzi dla

tego przypadku użycia wynosi jedynie około 58% początkowego rozmiaru. Użycie projekcji zasobu pozwala na osiągnięcie takiego samego wyniku jak przy żądaniu dostosowanym do przypadku użycia. Pokazuje to skuteczność tego mechanizmu w kontekście zmniejszenia rozmiaru przesyłanych danych.

Ważnym zagadnieniem jest również wpływ zastosowania przedstawionych rozwiązań na czas odpowiedzi oraz liczbę obsłużonych zapytań. Przeprowadzone testy zweryfikowały wpływ zminimalizowanego rozmiaru odpowiedzi na zwiększenie kosztu oraz wydłużenie czasu obsługi zapytania po stronie aplikacji serwerowej. Następujący rysunek przedstawia średni czas odpowiedzi w zależności od liczby użytkowników dla poszczególnych wariantów zastosowanych mechanizmów:



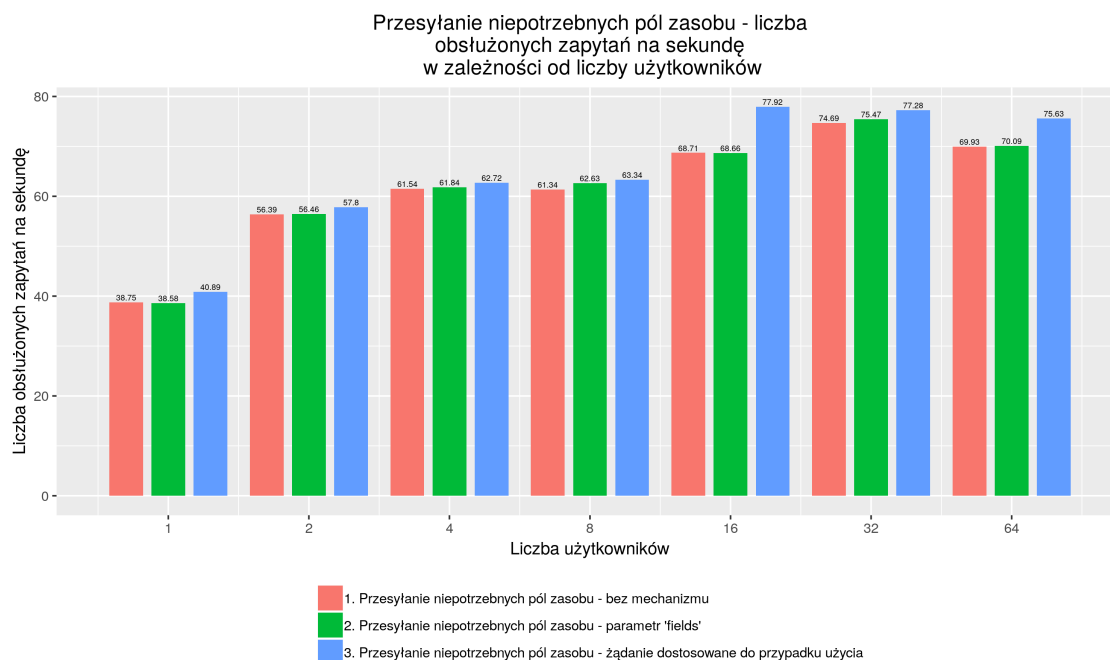
RYSUNEK 10.2: Przesyłanie niepotrzebnych pól zasobu – średni rozmiar odpowiedzi w zależności od liczby użytkowników

Otrzymane wyniki pokazują, że średni czas odpowiedzi rośnie wraz z liczbą użytkowników. Wynika to ze zwiększenia obciążenia serwera. Większa liczba równoległe przetwarzanych zapytań powoduje zmniejszenie przydzielonych zasobów na obsługę danego zapytania. Niezależnie od implementacji serwera i przyjętej strategii obsługi jednocześnie wielu zapytań wiąże się to z wydłużeniem średniego czasu odpowiedzi. Dla mniejszej liczby użytkowników różnice w uzyskanym wyniku dla poszczególnych mechanizmów są pomijalne. Widać jednak, że przy większej liczbie użytkowników najlepszy czas jest uzyskiwany dla zapytania dostosowanego do przypadku użycia. Wynika to między innymi z faktu, że dla tego przypadku koszt obsługi zapytania przez serwer jest najmniejszy – mniejsza niż w przypadku braku optymalizacji ilość danych jest poddawana mapowaniu i serializacji. Nie ma również miejsca analiza przekazywanego parametru „*fields*”, używanego w celu projekcji zasobu.

Jak stwierdzono na podstawie rys. 10.1 dla obydwu proponowanych rozwiązań problemu przesyłania niepotrzebnych pól zasobu uzyskano taką samą optymalizację wielkości odpowiedzi. Mniejszy rozmiar odpowiedzi powoduje zmniejszenie czasu przesłania jej przez Internet, co również wpływa na uzyskany czas odpowiedzi. Mechanizm projekcji zasobu wiąże się jednak z dodatkowym narzutem czasowym na analizę przekazanego parametru „*fields*” oraz zastosowanie go do odpowiedzi. Zwiększa to koszt obsługi zapytania przez serwer, przez co wyniki nie są tak dobre jak dla zapytania dostosowanego do przypadku użycia.

Rysunek 10.3 prezentuje liczbę obsłużonych zapytań na sekundę w zależności od liczby użytkowników dla poszczególnych wariantów zastosowanych rozwiązań:

Liczba obsłużonych zapytań rośnie wraz z liczbą użytkowników, ale tylko do pewnego momentu, stabilizując się, gdy liczba użytkowników wynosi 32. Wynika to z faktu, że dla mniejszej liczby użytkowników zasoby serwera nie są w pełni wykorzystywane. Powyżej pewnej liczby użytkowników zasoby serwera są wykorzystywane w pełni, więc nie jest on już w stanie uzyskać lepszego



RYSUNEK 10.3: Przesyłanie niepotrzebnych pól zasobu – liczba obsługiwanych zapytań na sekundę w zależności od liczby użytkowników

wyniku. Niezależnie od liczby użytkowników widać, że osiągnięta przez serwer wydajność jest różna w zależności od mechanizmu. Wynika to bezpośrednio z obserwacji dokonanych na podstawie rys. 10.2. Najlepsze wyniki są uzyskiwane dla zapytania dostosowanego do przypadku użycia, z kolei dla mechanizmu projekcji zasobu pojawia się jedynie niewielka poprawa względem braku optymalizacji.

10.1.2 Żądanie wielu zasobów

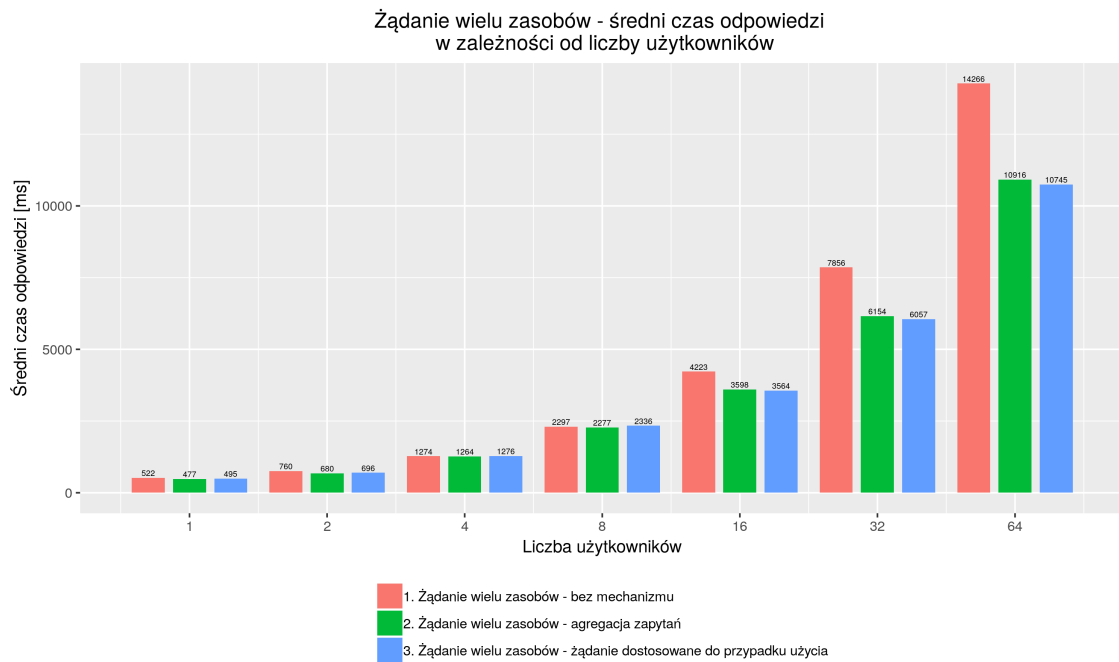
Przy żądaniu wielu zasobów głównym problemem jest konieczność utworzenia osobnego zapytania dla każdego z potrzebnych zasobów. Wiąże się to ze zwiększeniem sumarycznego rozmiaru odpowiedzi, kosztami po stronie klienta na utworzenie i obsługę wielu zapytań. Po stronie serwera, w zależności od zastosowanej architektury, obsługa wielu zapytań również może być kosztowna. Czynniki te przekładają się na wydajność systemu.

Rysunek 10.4 przedstawia średni czas odpowiedzi w zależności od liczby użytkowników dla poszczególnych wariantów zastosowanych mechanizmów.

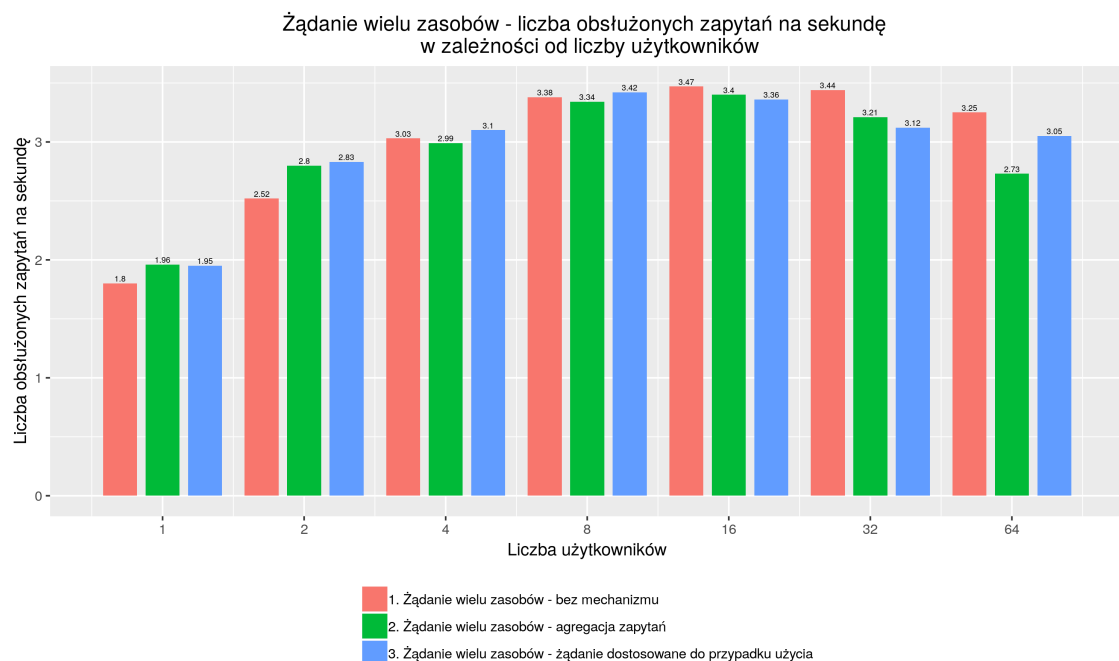
Dla większej liczby użytkowników średni czas odpowiedzi jest niższy przy zastosowaniu agregacji zapytań oraz żądania dostosowanego do przypadku użycia.

Na rys. 10.5 przedstawiono liczbę obsługiwanych zapytań na sekundę w zależności od liczby użytkowników dla poszczególnych wariantów zastosowanych mechanizmów. Z rysunku tego wynika, że mimo polepszenia średniego czasu odpowiedzi, liczba obsługiwanych zapytań na sekundę nie ulega poprawie, a nawet przy braku optymalizacji osiągnięte wyniki są nieznacznie lepsze. W testach przyjęto synchroniczny model wysyłania zapytań klienta, co oznacza, że każde zapytań jest wysyłane dopiero po otrzymaniu odpowiedzi, a uzyskany czas odpowiedzi obliczany na podstawie sumy ze wszystkich zapytań. Wynika z tego, że serwer w przypadku każdego mechanizmu będzie w danym momencie przetwarzać liczbę zapytań równą co najwyżej liczbie użytkowników. Również obsługa agregacji zapytań po stronie serwera jest wykonywana synchronicznie, jedno żądanie po drugim. Różnica w wydajności wynika więc jedynie z tworzenia wielu osobnych zapytań zamiast jednego. W tym przypadku przez klienta wymagane jest tylko siedem żądań, przez co różnice przy zastosowaniu różnych mechanizmów są niezauważalne i trudne do ustalenia.

Warte rozważenia jest również to, jak zastosowanie poszczególnych mechanizmów wpływa na rozmiar odpowiedzi. Rysunek 10.6 przedstawia średni rozmiar odpowiedzi w zależności od liczby użytkowników dla poszczególnych wariantów zastosowanych mechanizmów. Z rysunku wynika, że



RYSUNEK 10.4: Żądanie wielu zasobów – średni czas odpowiedzi w zależności od liczby użytkowników

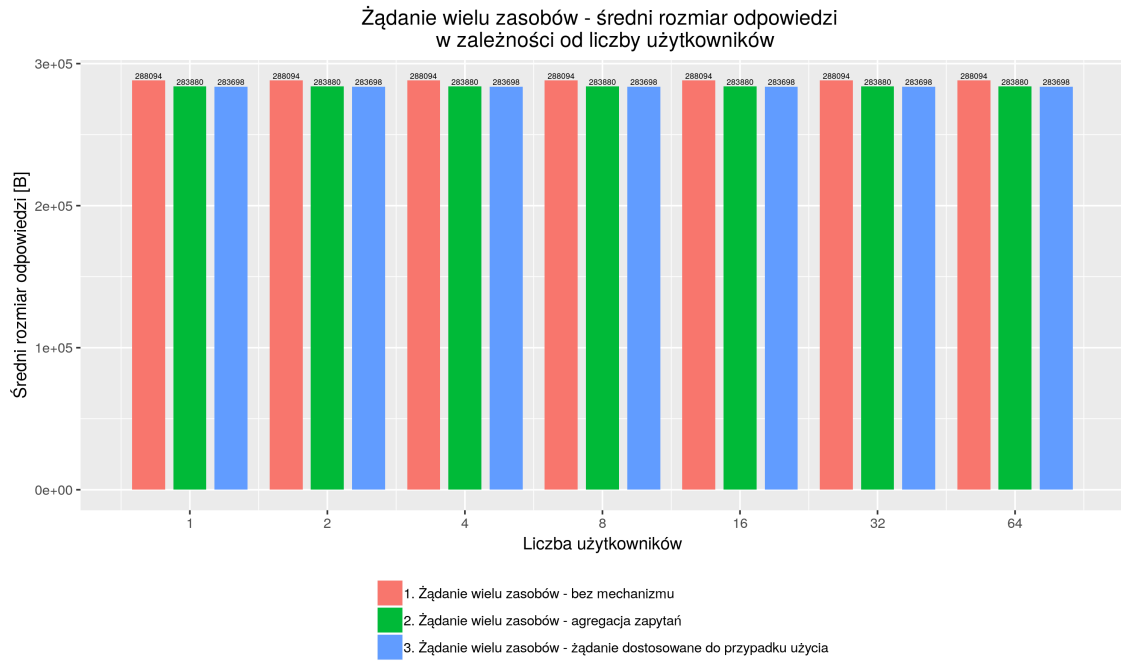


RYSUNEK 10.5: Żądanie wielu zasobów – liczba obsłużonych zapytań na sekundę w zależności od liczby użytkowników

różnice między poszczególnymi rozwiązaniami są znikome. Wynika to, podobnie jak w przypadku liczby obsłużonych zapytań na sekundę, z małej liczby żądań poddawanych agregacji.

10.1.3 Problem N+1 zapytań

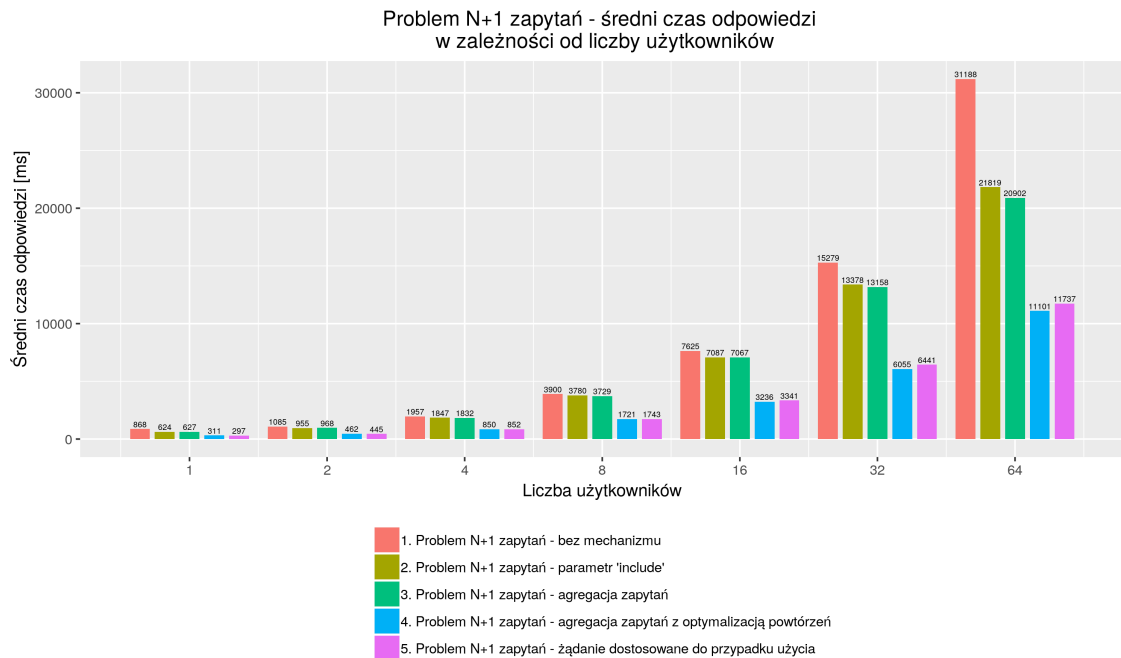
Problem ten jest w zasadzie pewnym przypadkiem specjalnym problemu żądania wielu zasobów. Fakt powiązania zasobów ze sobą pozwala na zastosowanie także innych mechanizmów optymalizacji. Na wydajność odpowiedzi, podobnie jak przy problemie żądania wielu zasobów,



RYСУNEK 10.6: Żądanie wielu zasobów – średni rozmiar odpowiedzi w zależności od liczby użytkowników

wpływa głównie potrzeba tworzenia wielu zapytań, osobno dla każdego żądania.

Rysunek 10.7 przedstawia średni czas odpowiedzi w zależności od liczby użytkowników dla poszczególnych wariantów zastosowanych mechanizmów.



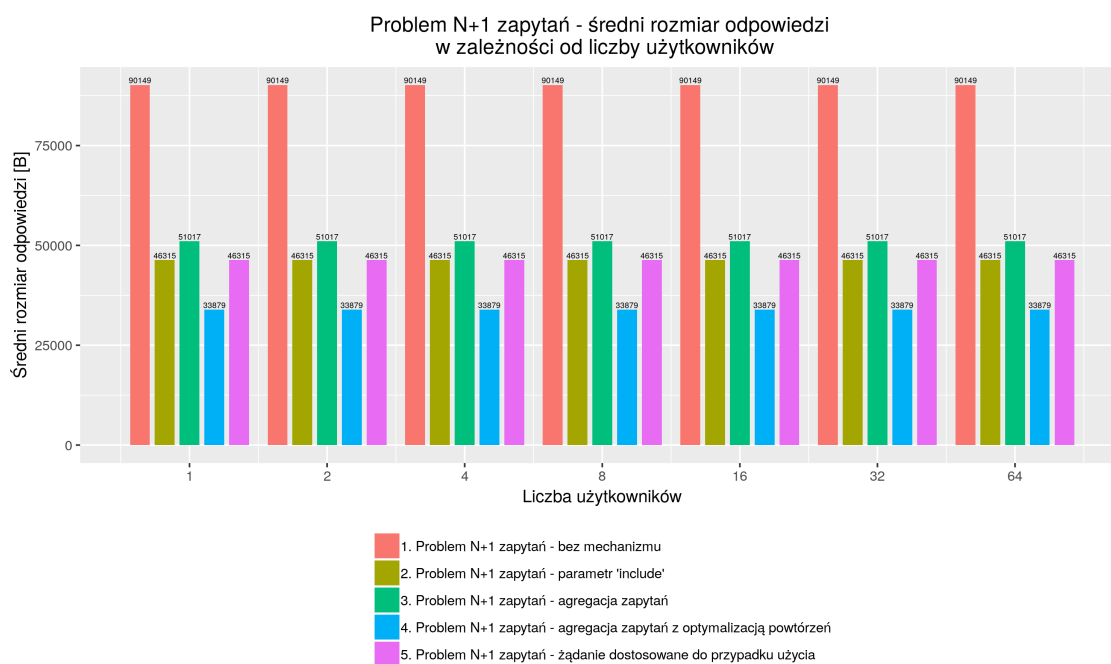
RYСУNEK 10.7: Problem N+1 zapytań - średni czas odpowiedzi w zależności od liczby użytkowników

Można zaobserwować duże różnice pomiędzy poszczególnymi rozwiązaniami, szczególnie dla większej liczby użytkowników. Zastosowanie połączenia zasobu powiązanego lub agregacji zapytań pozwala na uzyskanie już znacznej optymalizacji, ponieważ w tym przypadku znaczenie liczby osobnych zapytań wysyłanych przy braku optymalizacji jest znaczenie większe. Dla danych w

testowanej bazie danych, konieczne było wykonanie łącznie 63 zapytań, ponieważ liczba promocji w systemie wynosiła 62.

Jeszcze większa optymalizacja jest uzyskana dla agregacji zapytań z optymalizacją powtórzeń oraz dla żądania dostosowanego do przypadku użycia. W przypadku żądania dostosowanego do przypadku użycia również została zastosowana optymalizacja powtórzeń i z tego powodu wyniki są zbliżone. W przypadku agregacji zapytań z optymalizacją powtórzeń zostanie jednak przesłane mniej danych, ponieważ to po stronie klienta została podjęta decyzja o niewysyłaniu żądań o powtórzone kursy. Przy żądaniu dostosowanym do przypadku użycia w samej odpowiedzi pojawiają się duplikaty kursów, ponieważ odpowiedni kurs jest dołączany do każdej z promocji.

Różnicę w rozmiarze odpowiedzi widać na rys. 10.8.



RYСУNEK 10.8: Problem N+1 zapytań – średni rozmiar odpowiedzi w zależności od liczby użytkowników

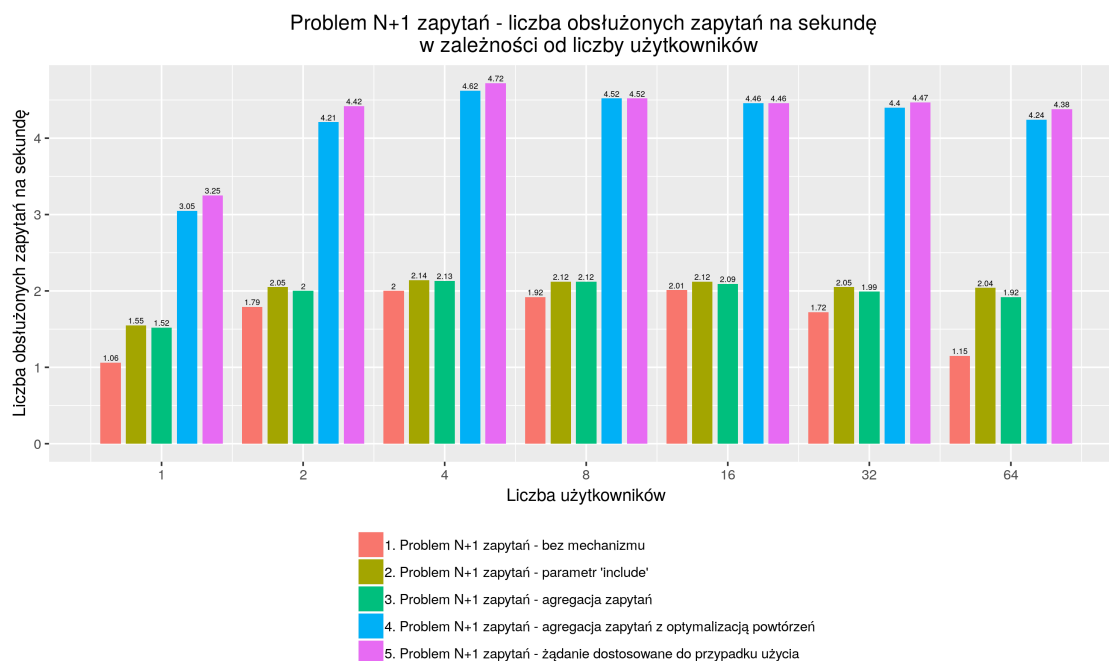
Czyste dane dla tego przypadku testowego, bez nagłówek i przy braku optymalizacji, mają rozmiar 48000B. Przy braku optymalizacji sumaryczny rozmiar odpowiedzi jest znacznie większy. Wynika to z narzutu na nagłówek HTTP oraz nagłówki niższych warstw sieciowych. Dodatkowo w testowanej aplikacji w każdej odpowiedzi przesyłany jest nagłówek HTTP „Set-Cookie”, zawierający między innymi klucz sesji użytkownika, którego rozmiar wynosi około 500B. Według rysunku łączny rozmiar nagłówek dla wszystkich odpowiedzi wynosi 46307B, co stanowi 49% całkowitego rozmiaru odpowiedzi. Dla pojedynczej odpowiedzi narzut na nagłówki wynosi więc około 735B. Przy większej liczbie zapytań problem ten ma więc coraz większe znaczenie.

W przypadku zastosowania mechanizmu połączenia zasobu powiązanego oraz dla zapytania dostosowanego do przypadku użycia rozmiar odpowiedzi wynosi jedynie 46381B, co stanowi liczbę nawet mniejszą niż dla czystych danych przy braku optymalizacji. Wynika to z tego, że dla tych rozwiązań zasób powiązany, czyli w tym przypadku kurs, jest wysyłany jako pole głównego zasobu (promocji). Dzięki temu nie ma potrzeby przesyłania identyfikatora powiązanego zasobu dwukrotnie, najpierw jako pole zasobu głównego, a następnie w osobnej odpowiedzi do zapytania o konkretny kurs. Jest to minimalna optymalizacja, jednak tłumaczy zmniejszenie rozmiaru poniżej 48000B. Głównym czynnikiem jest tu jednak uniknięcie wielokrotnego przesyłania nagłówek, szczególnie nagłówka HTTP „Set-Cookie”.

Dla agregacji zapytań rozmiar również jest znacznie zmniejszony względem braku mechanizmu. Rozmiar danych również wynosi 48000B, ale istnieje konieczność traktowania odpowiedzi przynajmniej częściowo tak, jakby dotyczyły niezależnych zapytań HTTP, w wyniku czego pojawia się pewien narzut na przesłanie potrzebnych danych. Jest on jednak znacznie mniejszy niż przy braku optymalizacji.

Podobnie jak w przypadku średniego czasu odpowiedzi, najlepszy wynik jest osiągnięty przy zastosowaniu agregacji zapytań z optymalizacją powtórzeń. W tym przypadku klient tworzy osobne zapytania o potrzebne kursy, dzięki czemu może zoptymalizować liczbę zapytań (a więc także odpowiedzi) wykluczając zduplikowane kursy. Liczba przesłanych danych będzie więc mniejsza, ponieważ możliwe jest uniknięcie zarówno narzutu na nagłówki, jak i niepotrzebnego przesyłania danych zduplikowanych.

Rysunek 10.9 przedstawia liczbę obsłużonych zapytań na sekundę w zależności od liczby użytkowników dla poszczególnych wariantów zastosowanych mechanizmów.



RYSUNEK 10.9: Problem N+1 zapytań – liczba obsłużonych zapytań na sekundę w zależności od liczby użytkowników

Podobnie jak przy średnim czasie oraz rozmiarze odpowiedzi wyniki potwierdzają skuteczność proponowanych mechanizmów. Szczególnie dobre rezultaty otrzymano dla agregacji zapytań z optymalizacją powtórzeń oraz żądania dostosowanego do przypadku życia. Mechanizmy te pozwalają na osiągnięcie nawet prawie czterokrotną poprawę liczby obsłużonych zapytań. Dla połączenia zasobu powiązanego uzyskiwane wyniki są nieznacznie lepsze niż dla agregacji zapytań. Niezależnie od liczby użytkowników obydwa te mechanizmy pozwalają jednak na optymalizację nawet do około 75%.

10.2 Analiza kosztów tworzenia i utrzymania systemu

W niniejszym podrozdziale przeprowadzono analizę kosztów tworzenia i utrzymania systemu dla każdego z proponowanych w pracy rozwiązań.

10.2.1 Żądanie dostosowane do konkretnego przypadku użycia.

Rozwiązanie to wymaga specyficznej implementacji dla każdego z przypadków użycia występujących w systemie. W praktyce jest ono możliwe do zastosowania jedynie w aplikacjach, dla których klient ich API jest znany i jest możliwe szczegółowe określenie jego wymagań. Co więcej, przy zmianie wymagań konieczna jest modyfikacja kodu zarówno aplikacji klienta, jak i serwerowej. Wiąże się to nie tylko ze zwiększeniem kosztu implementacji poszczególnych aplikacji, ale również ze znaczną potrzebą synchronizacji zespołów lub osób odpowiedzialnych za ich tworzenie.

10.2.2 Projekcja zasobu przy pomocy parametru „*fields*”.

Implementacja tego mechanizmu jest możliwa w sposób generyczny dla dowolnego żądania, dzięki czemu można go stosować bez dodatkowych kosztów przy tworzeniu i utrzymywaniu systemu. Dzięki ograniczeniu pól zasobu otrzymywanych przez klienta możliwe jest zmniejszenie kosztu implementacji po stronie aplikacji klienta, gdyż zanika potrzeba obsługi nadmiarowych pól. Co więcej, używanie tego rozwiązania może ograniczyć potrzebę modyfikacji aplikacji klienta w przypadku zmiany kontraktu poszczególnych zasobów. Gdy zmiany w reprezentacji zasobu dotyczą innych pól niż te używane przez klienta i wyszczególnione w ramach mechanizmu projekcji zasobu, nie ma potrzeby dostosowywania aplikacji klienta do tych zmian.

10.2.3 Połączenie zasobu powiązanego przy pomocy parametru „*include*”.

Mechanizm połączenia zasobu powiązanego może zostać zaimplementowany w sposób generyczny w kodzie infrastruktury, jednak konieczna jest jego odpowiednia konfiguracja dla każdego zasobu, w ramach którego ma zostać użyty. Stosowanie tego rozwiązania wiąże się więc z dodatkowymi kosztami po stronie aplikacji serwerowej. W aplikacji klienta użycie parametru „*include*” powoduje uproszczenie kodu, ponieważ zanika potrzeba obsługi dodatkowych zapytań o powiązane zasoby.

10.2.4 Agregacja zapytań

Obsługa agregacji zapytań po stronie serwera nie wpływa na koszt tworzenia i utrzymania systemu, ponieważ odbywa się w sposób generyczny w kodzie infrastruktury. Po stronie klienta stosowanie tego mechanizmu wiąże się dążeniem do umieszczania kodu obsługi zapytań w jednym miejscu w ramach każdego przypadku użycia. Trudno ocenić jak takie ograniczenie wpływa na koszty implementacji i czytelność kodu, ponieważ jest to kwestia subiektywna. Poza tym jednak użycie agregacji zapytań jest transparentne w aplikacji klienta i nie wpływa na koszt tworzenia i utrzymania systemu.

10.3 Wnioski

Analiza wyników testów wydajnościowych oraz kosztów tworzenia i utrzymania systemów doprowadziły do sformułowania następujących wniosków:

- Mechanizm projekcji zasobu pozwala na uzyskanie znacznej optymalizacji rozmiaru odpowiedzi, nie wpływa jednak znacząco na wydajność systemu w kwestii średniego czasu odpowiedzi oraz liczby obsługiwanych zapytań. Jego stosowanie wpływa korzystnie na koszt tworzenia i utrzymania systemu.
- Mechanizm połączenia powiązanego zasobu pozwala na dużą optymalizację zarówno rozmiaru, czasu odpowiedzi, jak i liczby obsługiwanych zapytań. Jego zastosowanie jest jednak ograniczone do przypadków, gdy zasoby są ze sobą bezpośrednio powiązane. Ogranicza też możliwość optymalizacji po stronie aplikacji klienta. Jego zastosowanie wpływa na koszt tworzenia i utrzymania systemu, zwiększając nakład na aplikację serwerową, ale upraszczając kod aplikacji klienta.
- Zastosowanie agregacji zapytań skutkuje znaczną optymalizacją rozmiaru i czasu odpowiedzi oraz liczby obsługiwanych zapytań. Jej użycie nie wpływa znacząco na koszt tworzenia i utrzymania systemu, choć nakłada pewne ograniczenia po stronie aplikacji klienta. Pozwala również na dalszą optymalizację po stronie klienta.
- Agregacja zapytań pozwala na rozwiązanie problemu $N+1$ zapytań mniejszym kosztem niż mechanizm połączenia powiązanego zasobu przy uzyskaniu podobnych efektów dla wydajności systemu. Przy dodatkowym nakładzie pracy pozwala na uzyskanie lepszych efektów dla wydajności systemu.
- Agregacja zapytań w łatwy sposób może zostać połączona z mechanizmem projekcji zasobu.
- Jednoczesne zastosowanie połączenia powiązanego zasobu oraz projekcji zasobu sprawia, że tworzenie zapytania jest mniej intuicyjne dla programisty oraz zwiększa złożoność obsługi zapytania po stronie serwera.

Rozdział 11

Podsumowanie i kierunek dalszych prac

W ramach pracy zaimplementowano bibliotekę wspierającą komunikację w modelu RPC. Biblioteka obsługuje mechanizmy rozwiązujące zdefiniowane w pracy problemy wydajnościowe przy komunikacji między aplikacjami w systemach rozproszonych. Przeprowadzono testy wydajnościowe oraz analizę wpływu zastosowania proponowanych mechanizmów na szybkość tworzenia i łatwość utrzymania systemu. Cel pracy został osiągnięty. Wyniki przeprowadzonych testów i analiz dały odpowiedź na wiele interesujących pytań związanych z rozważanymi przypadkami użycia.

Przedstawiona praca ma potencjał do dalszego rozwoju, zarówno w kierunku głębszej analizy proponowanych rozwiązań, jak i identyfikacji innych problemów komunikacji między aplikacjami oraz propozycji mechanizmów rozwiązujących te problemy.

W kontekście już przeanalizowanych problemów i mechanizmów ciekawe byłoby zbadanie ich wpływu na wydajność w systemie o architekturze mikroservisów [NSS14]. Wówczas mogłoby się okazać, że zalety wynikające z ograniczenia liczby zapytań zanikają w świetle możliwości rozproszonego przetwarzania zapytań na wielu węzłach. Można by jednak zaproponować inne implementacje przedstawionych mechanizmów, które lepiej wykorzystują zalety środowiska rozproszonego. Kolejną możliwością jest zbadanie użycia proponowanych rozwiązań razem ze wspieranymi przez protokół HTTP mechanizmami kompresji danych [FGM⁺99] oraz buforowania odpowiedzi [FGM⁺99]. Wpływ projekcji zasobu na zmniejszenie rozmiaru odpowiedzi mógłby być inny przy zastosowaniu kompresji danych. Z drugiej strony kompresja może dać lepsze skutki dla większej liczby danych przesyłanych w jednej odpowiedzi, co ma miejsce dla agregacji zapytań oraz przyłączenia powiązanych zasobów. Proponowane mechanizmy powodują, że odpowiedzi serwera są bardziej dostosowane do specyficznych potrzeb klienta. Może to negatywnie wpłynąć na zalety wynikające z buforowania odpowiedzi, gdzie zastosowanie bardziej ogólnego zapytania pozwoliłoby na użycie zbuforowanej odpowiedzi w wielu kontekstach.

W komunikacji między aplikacjami pojawiają się też inne problemy poza tymi wspomnianymi w pracy. Przykładem jest na przykład przesyłanie niepotrzebnych instancji zasobu, które mogłyby zostać pominięte już po stronie serwera poprzez dostarczenie mechanizmu filtrów. Ciekawym zagadnieniem jest również wysyłanie przez klienta wielokrotnie tego samego żądania API, lecz wywołanego z różnymi parametrami. Grupa takich zapytań tworzy zapytanie zbiorcze (bulk operation) [bul], które często może być obsłużone przez serwer w sposób znacznie wydajniejszy niż przy osobnej obsłudze poszczególnych zapytań. Wprowadzenie takiej optymalizacji wymaga zdefiniowania nowego żądania w API serwera, które może przyjąć wielokrotnie wymagane parametry. Z punktu widzenia funkcjonalności jest to taka sama operacja jak dla serii pojedynczych zapytań, lecz mimo to konieczne jest utworzenie dodatkowego żądania. Wykorzystując mechanizm agregacji zapytań można osiągnąć tę optymalizację bez definicji nowego żądania i zmieniania kontraktu między serwerem a klientem. Pozwala to również na bardziej generyczną implementację mechanizmu zapytań zbiorczych. Dalszy rozwój pracy mógłby zostać przeprowadzony w kierunku implementacji i analizy skuteczności takiego mechanizmu.

Założeniem w pracy była optymalizacja jedynie na poziomie komunikacji między aplikacjami. Przy odpowiednim powiązaniu warstw w architekturze aplikacji serwerowej jest jednak możliwa optymalizacja również na poziomie logiki biznesowej oraz dostępu do danych. Interesującym rozwinięciem pracy byłaby analiza wydajności takiej optymalizacji oraz związanych z nią kosztów przy tworzeniu i utrzymaniu systemu.

Literatura

- [Bie18] Matthias Biehl. *GraphQL API Design*. API-University Press, 2018.
- [BN84] Andrew D Birrell, Bruce Jay Nelson. Implementing remote procedure calls. *ACM Transactions on Computer Systems (TOCS)*, 2(1):39–59, 1984.
- [brm] Brmi - batched java rmi. <http://research.cs.vt.edu/vtspaces/brmi/>.
- [bul] Oracle - batch and bulk operations. <https://docs.oracle.com/en/cloud/paas/integration-cloud-service/cccdg/batch-and-bulk-operations.html>.
- [DM95] François-Nicola Demers, Jacques Malenfant. Reflection in logic, functional and object-oriented programming: a short comparative study. *Proceedings of the IJCAI*, wolumen 95, strony 29–38, 1995.
- [DRJ05] Gabriel Dos Reis, Jaakko Järvi. What is generic programming? *Library-Centric Software Design (LCSD'05)*, strona 1, 2005.
- [Fet11] Ian Fette. The websocket protocol. 2011.
- [FGM⁺99] Roy Fielding, Jim Gettys, Jeffrey Mogul, Henrik Frystyk, Larry Masinter, Paul Leach, Tim Berners-Lee. Hypertext transfer protocol-http/1.1, 1999.
- [gcp] Google cloud platform. <https://cloud.google.com/>.
- [gok] Gokoan. <https://www.gokoan.com/>.
- [goo] Arrow/monad comprehensions. <https://cloud.google.com/apis/design/>.
- [GSD⁺15] Andy Gill, Neil Sculthorpe, Justin Dawson, Aleksander Eskilson, Andrew Farmer, Mark Grebe, Jeffrey Rosenbluth, Ryan Scott, James Stanton. The remote monad design pattern. *ACM SIGPLAN Notices*, wolumen 50, strony 59–70. ACM, 2015.
- [HP17] Olaf Hartig, Jorge Pérez. An initial analysis of facebook’s graphql language. 2017.
- [jac] Fasterxml/jackson: Main portal page for the jackson project. <https://github.com/FasterXML/jackson>.
- [kto] Ktor - asynchronous web framework for kotlin. <https://ktor.io/>.
- [loc] Locust - a modern load testing framework. <https://locust.io/>.
- [Mar80] Christopher D Marlin. *Coroutines: a programming methodology, a language design and an implementation*. Number 95. Springer Science & Business Media, 1980.
- [Mas11] Mark Masse. *REST API Design Rulebook: Designing Consistent RESTful Web Service Interfaces*. Ó'Reilly Media, Inc.”, 2011.
- [MB11] Claudia Müller-Birn. *Architecutre of distributed systems*. 2011.
- [mic] Odata - the best way to rest. <https://www.odata.org/>.
- [mon] Arrow/monad comprehensions. https://arrow-kt.io/docs/patterns/monad_comprehensions/.
- [New15] Sam Newman. *Building microservices: designing fine-grained systems*. Ó'Reilly Media, Inc.”, 2015.
- [NPRI09] Nurzhan Nurseitov, Michael Paulson, Randall Reynolds, Clemente Izurieta. Comparison of json and xml data interchange formats: a case study. *Caine*, 9:157–162, 2009.
- [NR05] Cameron Newham, Bill Rosenblatt. *Learning the bash shell: Unix shell programming*. Ó'Reilly Media, Inc.”, 2005.

- [NSS14] Dmitry Namiot, Manfred Sneps-Sneppe. On micro-services architecture. *International Journal of Open Information Technologies*, 2(9):24–27, 2014.
- [OZK07] Cagil R Ozansoy, Aladin Zayegh, Akhtar Kalam. The real-time publisher/subscriber communication model for distributed substation systems. *IEEE transactions on power delivery*, 22(3):1411–1423, 2007.
- [PM01] Esmond Pitt, Kathy McNiff. *Java. rmi: The Remote Method Invocation Guide*. Addison-Wesley Longman Publishing Co., Inc., 2001.
- [rpc] How rpc works - microsoft docs.
<https://docs.microsoft.com/en-us/windows/win32/rpc/how-rpc-works>.
- [SB17] Stephen Samuel, Stefan Bocutiu. *Programming Kotlin*. Packt Publishing Ltd, 2017.
- [Sch01] Rüdiger Schollmeier. A definition of peer-to-peer networking for the classification of peer-to-peer architectures and applications. *Proceedings First International Conference on Peer-to-Peer Computing*, strongy 101–102. IEEE, 2001.
- [Shi10] Sang Shin. Introduction to json (javascript object notation). *Presentation www.javapassion.com*, 2010.
- [Ste14] Daniel Stenberg. Http2 explained. *Computer Communication Review*, 44(3):120–128, 2014.
- [VDKV00] Arie Van Deursen, Paul Klint, Joost Visser. Domain-specific languages: An annotated bibliography. *ACM Sigplan Notices*, 35(6):26–36, 2000.
- [Ven98] Bill Venners. *The java virtual machine*. McGraw-Hill, New York, 1998.
- [Wic16] Hadley Wickham. *ggplot2: elegant graphics for data analysis*. Springer, 2016.

© 2019 Marcin Piniarski

Instytut Informatyki, Wydział Informatyki
Politechnika Poznańska

Skład przy użyciu systemu L^AT_EX.

BibT_EX:

```
@mastersthesis{ key,
  author = "Marcin Piniarski",
  title = "{Analiza i optymalizacja mechanizmu komunikacji między aplikacjami w systemach
rozproszonych przy użyciu modelu RPC}",
  school = "Poznan University of Technology",
  address = "Pozna{\n}, Poland",
  year = "2019",
}
```